

自然语言处理（NLP）教程

第三节课：词向量与语言模型

ClimbAI

2025 年 9 月 6 日

目录

1 课程概述	4
1.1 学习目标	4
1.2 课程安排	4
2 词向量基础理论 (50 分钟)	4
2.1 1.1 分布式表示理论 (15 分钟)	4
2.1.1 传统词汇表示的局限性	4
2.1.2 分布式假设	5
2.1.3 词向量的优势	6
2.2 1.2 词向量的数学基础 (20 分钟)	6
2.2.1 向量空间模型	6
2.2.2 相似性度量	6
2.2.3 词汇关系的向量表示	6
2.3 1.3 词向量的评估方法 (15 分钟)	7
2.3.1 内在评估	7
2.3.2 外在评估	7
3 主流词向量算法 (70 分钟)	8
3.1 2.1 Word2Vec 算法 (25 分钟)	8
3.1.1 Word2Vec 的核心思想	8
3.1.2 Skip-gram 模型详解	8
3.2 2.2 GloVe 算法 (20 分钟)	9
3.2.1 GloVe 的核心思想	9
3.2.2 理论推导	9
3.2.3 共现矩阵	9
3.2.4 GloVe 目标函数	10
3.3 2.3 FastText 算法 (25 分钟)	10
3.3.1 FastText 的创新点	10
3.3.2 子词表示详解	11
3.3.3 FastText vs Word2Vec 对比	12
4 语言模型原理与实现 (60 分钟)	12
4.1 3.1 语言模型基础 (20 分钟)	12
4.1.1 语言模型的定义与重要性	12
4.1.2 应用场景详解	13
4.1.3 语言模型发展历程	13
4.1.4 评估指标详解	14
4.1.5 GloVe 算法实现	15
4.2 3.2 N-gram 语言模型 (15 分钟)	25

4.2.1	N-gram 模型原理	25
4.2.2	参数估计	26
4.3	3.3 神经网络语言模型 (25 分钟)	26
4.3.1	神经网络语言模型的优势	26
4.3.2	FastText 算法实现	26
4.3.3	前馈神经网络语言模型	36
4.3.4	循环神经网络语言模型	37
4.3.5	神经网络语言模型实现	38
5	总结与展望	62
5.1	本节课总结	62
6	课后练习	63
6.1	基础练习	63
7	参考资料	63
7.1	核心论文	63

1 课程概述

1.1 学习目标

知识目标 (Knowledge Objectives)

- 理解词向量的基本原理和分布式假设
- 掌握 Word2Vec、GloVe、FastText 等词向量算法
- 了解语言模型的概念和发展历程
- 理解从 n-gram 到神经网络语言模型的演进

技能目标 (Skill Objectives)

- 能够训练和使用不同类型的词向量
- 掌握词向量的评估和可视化方法
- 学会构建简单的神经网络语言模型
- 能够应用词向量解决实际 NLP 问题

能力目标 (Competency Objectives)

- 能够根据任务特点选择合适的词向量方法
- 具备设计和优化词向量训练流程的能力
- 能够分析词向量的质量和偏差问题
- 培养对语言模型发展趋势的理解和判断力

1.2 课程安排

表 1: 第三节课时间安排

时间段	内容	时长
第 1 部分	词向量基础理论	50 分钟
第 2 部分	主流词向量算法	70 分钟
第 3 部分	语言模型原理与实现	60 分钟
总计		180 分钟

2 词向量基础理论 (50 分钟)

2.1 1.1 分布式表示理论 (15 分钟)

2.1.1 传统词汇表示的局限性

传统的词汇表示方法（如 one-hot 编码）存在以下问题：

One-hot 表示

词汇: 猫

	0	0	1	0	0	...	0	0	0	0	
--	---	---	---	---	---	-----	---	---	---	---	--

维度高、稀疏

无语义关系

词向量表示

词汇: 猫

0.3	0.1	0.8	0.5	0.2	0.9	0.1	0.6	0.4	...	0.2	0.5	0.6	0.1	0.3	0.7	0.8	0.4	0.3	
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	--

维度低、密集

包含语义

图 1: 传统表示 vs 词向量表示

One-hot 编码的问题:

1. **维度爆炸**: 词汇表大小 $|V|$ 通常为 10 万-100 万
2. **稀疏性**: 每个向量只有一个位置为 1, 其余为 0
3. **无语义关系**: 任意两个词向量的内积为 0
4. **存储低效**: 需要大量内存存储稀疏矩阵

2.1.2 分布式假设

分布式假设 (Distributional Hypothesis) 是词向量的理论基础:

"You shall know a word by the company it keeps."

”词汇的含义由其上下文决定。”

— J.R. Firth (1957)

核心理念:

- 语义相似的词汇倾向于出现在相似的上下文中
- 通过统计上下文信息可以学习词汇的语义表示
- 词汇之间的关系可以通过向量运算来体现

数学表述: 如果词汇 w_i 和 w_j 语义相似, 那么它们的上下文分布 $P(context|w_i)$ 和 $P(context|w_j)$ 应该相似。

2.1.3 词向量的优势

表 2: 词向量 vs 传统表示方法

特性	One-hot 编码	词向量
维度	$ V $ (10 万 +)	50-1000
密度	极稀疏	密集
语义关系	无	有
存储效率	低	高
计算效率	低	高
泛化能力	无	强

2.2 1.2 词向量的数学基础 (20 分钟)

2.2.1 向量空间模型

词向量将词汇映射到连续的向量空间中：

$$f: W \rightarrow \mathbb{R}^d \quad (1)$$

其中 W 是词汇表， d 是向量维度（通常 $d \ll |W|$ ）。

向量空间的性质：

- 相似性度量：使用余弦相似度或欧几里德距离
- 线性关系：词汇关系可以通过向量运算表达
- 聚类特性：相似词汇在空间中聚集

2.2.2 相似性度量

1. 余弦相似度

$$\text{cosine}(\vec{v}_i, \vec{v}_j) = \frac{\vec{v}_i \cdot \vec{v}_j}{|\vec{v}_i| \cdot |\vec{v}_j|} = \frac{\sum_{k=1}^d v_{i,k} \cdot v_{j,k}}{\sqrt{\sum_{k=1}^d v_{i,k}^2} \cdot \sqrt{\sum_{k=1}^d v_{j,k}^2}} \quad (2)$$

2. 欧几里德距离

$$\text{distance}(\vec{v}_i, \vec{v}_j) = \sqrt{\sum_{k=1}^d (v_{i,k} - v_{j,k})^2} \quad (3)$$

2.2.3 词汇关系的向量表示

词向量最著名的特性是能够通过向量运算表达词汇关系：

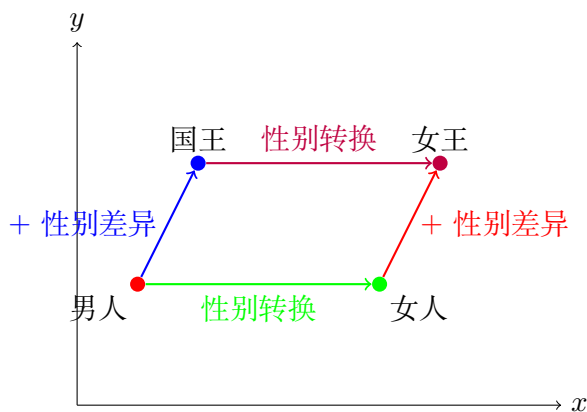


图 2: 词向量的线性关系示例

经典的类比关系:

$$\vec{king} - \vec{man} + \vec{woman} \approx \vec{queen} \quad (4)$$

$$\vec{Paris} - \vec{France} + \vec{Italy} \approx \vec{Rome} \quad (5)$$

$$\vec{walked} - \vec{walk} + \vec{swim} \approx \vec{swam} \quad (6)$$

2.3 1.3 词向量的评估方法 (15 分钟)

2.3.1 内在评估

内在评估直接测试词向量的质量，不依赖于具体的下游任务。

1. **词汇相似性任务**使用人工标注的词汇对相似性分数与词向量计算的相似性进行比较。

表 3: 词汇相似性数据集示例

词汇 1	词汇 2	人工评分	向量相似度
汽车	轿车	9.2	0.89
狗	猫	7.5	0.73
苹果	橙子	6.8	0.65
书	电脑	2.1	0.23

评估指标: Spearman 相关系数

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)} \quad (7)$$

2. **词汇类比任务**测试词向量是否能够正确完成 $a : b :: c : ?$ 的类比推理。

评估方法:

$$\vec{d}^* = \arg \max_{\vec{d} \in V, d \neq a, b, c} \text{cosine}(\vec{b} - \vec{a} + \vec{c}, \vec{d}) \quad (8)$$

2.3.2 外在评估

外在评估通过下游任务的性能来评估词向量的质量。

常用下游任务:

- **文本分类**：情感分析、主题分类
- **命名实体识别**：人名、地名、机构名识别
- **词性标注**：判断词汇的语法角色
- **语义角色标注**：识别句子中的语义关系

3 主流词向量算法 (70 分钟)

3.1 2.1 Word2Vec 算法 (25 分钟)

3.1.1 Word2Vec 的核心思想

Word2Vec 基于神经网络来学习词向量，有两种主要架构：

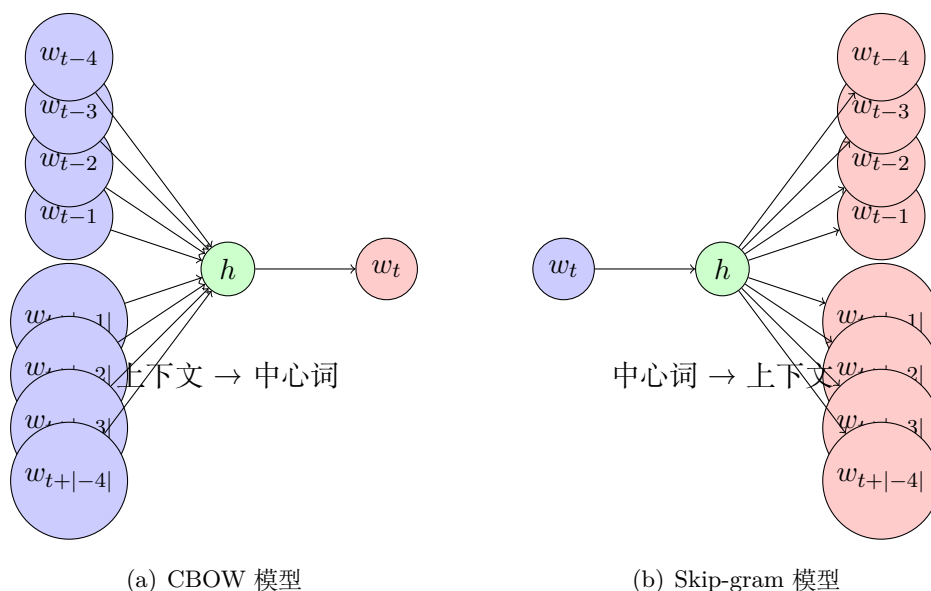


图 3: Word2Vec 的两种架构

3.1.2 Skip-gram 模型详解

Skip-gram 模型是 Word2Vec 中更常用的架构，目标是通过中心词预测上下文词汇。

模型目标：最大化以下似然函数：

$$L = \prod_{w \in W} \prod_{c \in C(w)} P(c|w) \quad (9)$$

其中 W 是语料库中的所有词汇， $C(w)$ 是词汇 w 的上下文窗口。

概率计算：使用 softmax 函数计算条件概率：

$$P(w_O|w_I) = \frac{\exp(\vec{v}_{w_O}^T \vec{v}_{w_I})}{\sum_{w=1}^W \exp(\vec{v}_w^T \vec{v}_{w_I})} \quad (10)$$

训练优化：由于 softmax 的分母需要对整个词汇表求和，计算复杂度为 $O(|V|)$ ，实际应用中使用了负采样优化技术。

3.2 2.2 GloVe 算法 (20 分钟)

3.2.1 GloVe 的核心思想

GloVe (Global Vectors) 结合了全局矩阵分解和局部上下文窗口的优点。

基本思想：词向量的内积应该等于词汇共现概率的对数：

$$\vec{w}_i^T \vec{w}_j + b_i + b_j = \log(X_{ij}) \quad (11)$$

其中：

- X_{ij} : 词汇 i 和 j 的共现次数
- b_i, b_j : 偏置项

3.2.2 理论推导

GloVe 的理论基础来自共现概率比值的分析。考虑三个词汇： i (ice), j (steam), k (solid/-gas/water/fashion)

表 4: 共现概率比值示例

$P(k i)/P(k j)$	$k=\text{solid}$	$k=\text{gas}$	$k=\text{water}$
比值	8.9	0.085	1.36

比值包含了重要的语义信息：

- 当 k 与 i 相关但与 j 无关时，比值很大
- 当 k 与 j 相关但与 i 无关时，比值很小
- 当 k 与两者都相关或都无关时，比值接近 1

3.2.3 共现矩阵

GloVe 首先构建全局词汇共现矩阵：

我	0	5	3	...	
喜欢	5	0	8	...	
机器	3	8	0	...	
...					

X_{ij} = 词汇 i 和 j 的共现次数

我 喜欢 机器 ...

图 4: 词汇共现矩阵示例

3.2.4 GloVe 目标函数

加权最小二乘目标函数：

$$J = \sum_{i,j=1}^V f(X_{ij})(\vec{w}_i^T \tilde{\vec{w}}_j + b_i + \tilde{b}_j - \log X_{ij})^2 \quad (12)$$

其中权重函数：

$$f(x) = \begin{cases} (x/x_{max})^\alpha & \text{if } x < x_{max} \\ 1 & \text{otherwise} \end{cases} \quad (13)$$

权重函数的设计原理：

- 防止高频词对占主导地位
- 给予罕见词对适当的权重
- 避免对数运算中的数值问题
- 通常设置 $x_{max} = 100$, $\alpha = 0.75$

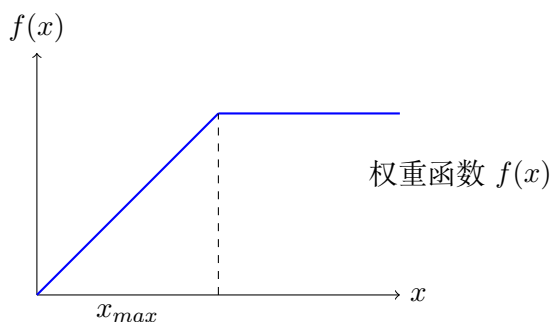


图 5: GloVe 权重函数

3.3 2.3 FastText 算法 (25 分钟)

3.3.1 FastText 的创新点

FastText 是 Word2Vec 的扩展，主要创新在于利用子词信息来处理词汇变形和未登录词问题。

核心思想：

- 将词汇分解为字符 n-gram 的集合
- 词向量 = 子词向量的平均
- 能够为训练中未见过的词汇生成向量
- 特别适合形态丰富的语言（如芬兰语、土耳其语、中文）

3.3.2 子词表示详解

字符 n-gram 提取原理:

对于中文词汇”机器学习”, 提取 2-4gram:

- 2-gram: < 机, 机器, 器学, 学习, 习 >
- 3-gram: < 机器, 机器学, 器学习, 学习 >
- 4-gram: < 机器学, 机器学习, 器学习 >

其中 < 和 > 是边界符号, 用于区分词的开始和结束。

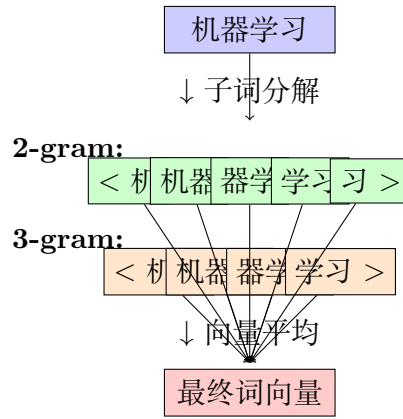


图 6: FastText 子词表示方法详解

子词提取算法:

Algorithm 1 子词提取算法

Require: 词汇 w , 最小 n-gram 长度 $\min_n$, 最大 n-gram 长度 $\max_n$

Ensure: 子词集合 S

```
1:  $w' \leftarrow < + w + >$  ▷ 添加边界符号
2:  $S \leftarrow \{\}$ 
3: for  $n = \min\_n$  to  $\min(|w'|, \max\_n)$  do
4:   for  $i = 0$  to  $|w'| - n$  do
5:      $subword \leftarrow w'[i : i + n]$ 
6:      $S \leftarrow S \cup \{subword\}$ 
7:   end for
8: end for
9:  $S \leftarrow S \cup \{w'\}$  ▷ 添加完整词汇
10: return  $S$ 
```

词向量计算:

$$\vec{w} = \frac{1}{|S_w|} \sum_{s \in S_w} \vec{z}_s \quad (14)$$

其中:

- S_w : 词汇 w 的所有子词集合
- z_s : 子词 s 的向量表示
- 包括词汇本身作为一个特殊的”子词”

3.3.3 FastText vs Word2Vec 对比

表 5: FastText 与 Word2Vec 详细对比

特性	Word2Vec	FastText
OOV 处理	无法处理未登录词 返回零向量或错误	可生成 OOV 词向量 基于子词信息推断
形态变化	每个词形独立学习 ”running”, ”runs” 无关联	共享子词信息 能识别词根关系
拼写错误	完全无法识别 ”computr” 无法处理	有一定鲁棒性 可基于”comput” 等推断
罕见词	向量质量差 训练数据不足	利用子词提升质量 子词在其他词中出现
训练速度	快 $O(V)$	相对较慢 $O(V + S)$
存储需求	小, 仅存词向量 $ V \times d$ 参数	大, 需存储子词向量 $(V + S) \times d$ 参数
语言适用性	适合孤立语言 英语效果好	适合黏着语、屈折语 芬兰语、土耳其语优势明显

4 语言模型原理与实现 (60 分钟)

4.1 3.1 语言模型基础 (20 分钟)

4.1.1 语言模型的定义与重要性

语言模型是计算文本序列概率的模型, 它试图回答”这个句子在某种语言中出现的可能性有多大?” 的问题。

数学定义: 给定词汇序列 $w_1, w_2, \dots, w_n$, 语言模型计算联合概率:

$$P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i | w_1, w_2, \dots, w_{i-1}) \quad (15)$$

这个定义基于概率论的链式法则。

语言模型的核心任务:

1. **概率估计:** 计算给定文本序列的概率
2. **文本生成:** 根据上下文生成下一个最可能的词

- 3. **序列评估**：比较不同序列的合理性
- 4. **补全预测**：预测缺失的词汇或短语

4.1.2 应用场景详解

表 6: 语言模型应用场景

应用领域	具体任务	模型作用
机器翻译	译文评估	判断翻译结果流畅性
	词序调整	选择最佳词汇顺序
	歧义消解	在多种翻译中选择
语音识别	候选排序	在多个识别结果中选择
	错误纠正	基于语言知识纠错
	语音合成	生成自然的语音文本
文本生成	自动写作	生成连贯的文章
	对话系统	产生合理的回复
	创意写作	协助创作诗歌、故事
信息检索	查询扩展	生成相关查询词
	相关性评估	判断文档与查询相关性

4.1.3 语言模型发展历程

统计 N-gram 模型 (1950s-1990s) 马尔可夫假设	神经网络语言模型 (2003-2010s) 分布式表示	循环神经网络语言模型 (2010s) 序列建模	Transformer 语言模型 (2017-) 注意力机制
离散符号 稀疏表示 计算简单	连续向量 密集表示 固定上下文	变长序列 递归结构 梯度问题	并行计算 长距离依赖 预训练模型

图 7: 语言模型发展历程

每个阶段的特点：

1. 统计 N-gram 时代 (1950s-1990s)

- **优点**：简单直观，计算高效，可解释性强
- **缺点**：数据稀疏，维度灾难，缺乏语义理解
- **代表**：Katz 回退模型，修正的 Kneser-Ney 平滑

2. 神经网络时代 (2003-2010s)

- **优点**：连续表示，参数共享，泛化能力强

- **缺点：**固定窗口大小，计算复杂度高
- **代表：**Bengio 神经概率语言模型

3. 循环网络时代 (2010s)

- **优点：**可处理变长序列，理论上可建模无限长依赖
- **缺点：**梯度消失/爆炸，无法并行训练
- **代表：**LSTM 语言模型，GRU 语言模型

4. 注意力机制时代 (2017-)

- **优点：**并行计算，长距离依赖，强大的表示能力
- **缺点：**计算复杂度高，需要大量数据
- **代表：**GPT, BERT, T5, GPT-3/4

4.1.4 评估指标详解

1. 困惑度 (Perplexity)

困惑度是语言模型最重要的评估指标：

$$PP(W) = P(w_1, w_2, \dots, w_N)^{-\frac{1}{N}} = \sqrt[N]{\frac{1}{P(w_1, w_2, \dots, w_N)}} \quad (16)$$

等价地，可以用交叉熵表示：

$$PP(W) = 2^{H(W)} \quad (17)$$

其中交叉熵为：

$$H(W) = -\frac{1}{N} \sum_{i=1}^N \log_2 P(w_i | w_1, \dots, w_{i-1}) \quad (18)$$

困惑度的直观理解：

- 困惑度表示模型在每个位置平均”困惑”的选择数量
- 如果词汇表大小为 V，随机模型的困惑度为 V
- 完美模型的困惑度为 1
- 困惑度越低，模型性能越好

2. 位级别困惑度

对于基于字符的模型：

$$BPC = \frac{H(W)}{\log_2(2)} = H(W) \quad (19)$$

3. BLEU 分数 (生成任务)

对于文本生成任务，使用 BLEU 评估生成质量：

$$BLEU = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right) \quad (20)$$

其中 p_n 是 n-gram 精确率， BP 是简洁性惩罚因子。

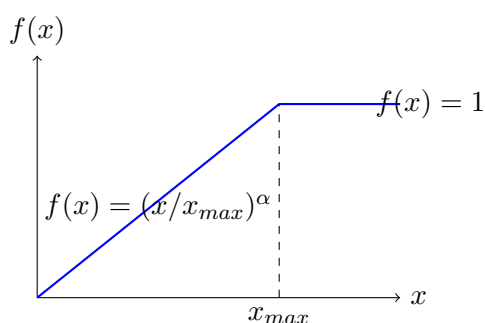


图 8: GloVe 权重函数

4.1.5 GloVe 算法实现

```
1 import numpy as np
2 from collections import defaultdict
3 import random
4 from scipy.sparse import dok_matrix
5 import matplotlib.pyplot as plt
6
7 class GloVe:
8     def __init__(self, vector_size=100, x_max=100, alpha=0.75, learning_rate=0.05):
9         """
10         GloVe 模型
11
12         Args:
13             vector_size: 词向量维度
14             x_max: 权重函数的最大值
15             alpha: 权重函数的指数
16             learning_rate: 学习率
17         """
18         self.vector_size = vector_size
19         self.x_max = x_max
20         self.alpha = alpha
21         self.learning_rate = learning_rate
22
23         # 词汇表
24         self.word2idx = {}
25         self.idx2word = {}
26         self.vocab_size = 0
27
28         # 共现矩阵
29         self.cooccurrence_matrix = None
```

```

30
31 # 模型参数
32 self.W = None # 主要词向量
33 self.W_tilde = None # 上下文词向量
34 self.b = None # 主要词偏置
35 self.b_tilde = None # 上下文词偏置
36
37 # 训练历史
38 self.loss_history = []
39
40 def build_vocab(self, sentences):
41     """构建词汇表"""
42     vocab = set()
43     for sentence in sentences:
44         vocab.update(sentence)
45
46     self.word2idx = {word: idx for idx, word in enumerate(sorted(vocab))}
47     self.idx2word = {idx: word for word, idx in self.word2idx.items()}
48     self.vocab_size = len(vocab)
49
50     print(f"词汇表大小: {self.vocab_size}")
51
52 def build_cooccurrence_matrix(self, sentences, window_size=5):
53     """构建共现矩阵"""
54     print("构建共现矩阵...")
55
56     # 使用稀疏矩阵存储
57     self.cooccurrence_matrix = dok_matrix((self.vocab_size, self.vocab_size))
58
59     total_pairs = 0
60     for sentence in sentences:
61         # 转换为词汇索引
62         indices = [self.word2idx[word] for word in sentence
63                    if word in self.word2idx]
64
65         # 计算共现
66         for i, center_idx in enumerate(indices):
67             for j, context_idx in enumerate(indices):
68                 if i != j:
69                     distance = abs(i - j)
70                     if distance <= window_size:
71                         # 距离加权: 距离越近权重越大
72                         weight = 1.0 / distance
73                         self.cooccurrence_matrix[center_idx, context_idx] +=
74                             weight
75                         total_pairs += 1
76
77     print(f"共现矩阵形状: {self.cooccurrence_matrix.shape}")
78     print(f"非零元素数量: {self.cooccurrence_matrix.nnz}")

```



```

78     print(f"处理的词对数量: {total_pairs}")
79
80     # 矩阵对称化 (可选)
81     self.cooccurrence_matrix = (self.cooccurrence_matrix +
82                                 self.cooccurrence_matrix.T) / 2
83
84     def _weight_func(self, x):
85         """GloVe 权重函数"""
86         if x < self.x_max:
87             return (x / self.x_max) ** self.alpha
88         else:
89             return 1.0
90
91     def _compute_loss(self, cooc_items):
92         """计算总损失"""
93         total_loss = 0
94         for (i, j), x_ij in cooc_items:
95             if x_ij > 0:
96                 # 计算预测值
97                 prediction = (np.dot(self.W[i], self.W_tilde[j]) +
98                               self.b[i] + self.b_tilde[j])
99
100                 # 计算损失
101                 diff = prediction - np.log(x_ij)
102                 weight = self._weight_func(x_ij)
103                 loss = weight * (diff ** 2)
104                 total_loss += loss
105
106         return total_loss
107
108     def train(self, epochs=50, verbose=True):
109         """训练 GloVe 模型"""
110         print(f"开始训练 GloVe 模型, 共 {epochs} 轮...")
111
112         # 初始化参数
113         scale = 1.0 / self.vector_size
114         self.W = np.random.uniform(-scale, scale, (self.vocab_size, self.vector_size))
115         self.W_tilde = np.random.uniform(-scale, scale, (self.vocab_size, self.vector_size))
116         self.b = np.random.uniform(-scale, scale, self.vocab_size)
117         self.b_tilde = np.random.uniform(-scale, scale, self.vocab_size)
118
119         # 获取共现矩阵的非零元素
120         cooc_items = list(self.cooccurrence_matrix.items())
121
122         # 初始损失
123         if verbose:
124             initial_loss = self._compute_loss(cooc_items)

```

```

125         print(f"初始损失: {initial_loss:.2f}")
126
127     for epoch in range(epochs):
128         epoch_loss = 0
129         random.shuffle(cooc_items)
130
131         for (i, j), x_ij in cooc_items:
132             if x_ij > 0:
133                 # 计算预测值
134                 prediction = (np.dot(self.W[i], self.W_tilde[j]) +
135                               self.b[i] + self.b_tilde[j])
136
137                 # 计算损失和梯度
138                 diff = prediction - np.log(x_ij)
139                 weight = self._weight_func(x_ij)
140
141                 # 计算梯度
142                 grad_factor = weight * diff * self.learning_rate
143
144                 # 更新参数
145                 # 词向量梯度
146                 grad_w_i = grad_factor * self.W_tilde[j]
147                 grad_w_tilde_j = grad_factor * self.W[i]
148
149                 # 偏置梯度
150                 grad_b_i = grad_factor
151                 grad_b_tilde_j = grad_factor
152
153                 # 参数更新
154                 self.W[i] -= grad_w_i
155                 self.W_tilde[j] -= grad_w_tilde_j
156                 self.b[i] -= grad_b_i
157                 self.b_tilde[j] -= grad_b_tilde_j
158
159                 epoch_loss += weight * (diff ** 2)
160
161         # 记录损失
162         self.loss_history.append(epoch_loss)
163
164         # 学习率衰减
165         if epoch > 0 and epoch % 10 == 0:
166             self.learning_rate *= 0.95
167
168         if verbose and epoch % 10 == 0:
169             print(f"Epoch {epoch}, Loss: {epoch_loss:.2f}, LR: {self.
170                   learning_rate:.4f}")
171
172     print("训练完成! ")

```

```

173 def get_vector(self, word):
174     """获取词向量（主要向量和上下文向量的平均）"""
175     if word in self.word2idx:
176         idx = self.word2idx[word]
177         return (self.W[idx] + self.W_tilde[idx]) / 2
178     return None
179
180 def get_word_vector(self, word):
181     """获取主要词向量"""
182     if word in self.word2idx:
183         idx = self.word2idx[word]
184         return self.W[idx]
185     return None
186
187 def get_context_vector(self, word):
188     """获取上下文词向量"""
189     if word in self.word2idx:
190         idx = self.word2idx[word]
191         return self.W_tilde[idx]
192     return None
193
194 def similarity(self, word1, word2):
195     """计算词汇相似度"""
196     vec1 = self.get_vector(word1)
197     vec2 = self.get_vector(word2)
198
199     if vec1 is None or vec2 is None:
200         return 0
201
202     # 余弦相似度
203     norm1 = np.linalg.norm(vec1)
204     norm2 = np.linalg.norm(vec2)
205
206     if norm1 == 0 or norm2 == 0:
207         return 0
208
209     return np.dot(vec1, vec2) / (norm1 * norm2)
210
211 def most_similar(self, word, topn=10):
212     """找到最相似的词汇"""
213     if word not in self.word2idx:
214         return []
215
216     word_vec = self.get_vector(word)
217     similarities = []
218
219     for other_word in self.word2idx:
220         if other_word != word:
221             sim = self.similarity(word, other_word)

```

```

222         similarities.append((other_word, sim))
223
224     similarities.sort(key=lambda x: x[1], reverse=True)
225     return similarities[:topn]
226
227 def analogy(self, word_a, word_b, word_c, topn=1):
228     """词汇类比"""
229     try:
230         vec_a = self.get_vector(word_a)
231         vec_b = self.get_vector(word_b)
232         vec_c = self.get_vector(word_c)
233
234         if vec_a is None or vec_b is None or vec_c is None:
235             return []
236
237         # 计算类比向量
238         analogy_vec = vec_b - vec_a + vec_c
239
240         # 找最相似的词汇
241         similarities = []
242         for word in self.word2idx:
243             if word not in [word_a, word_b, word_c]:
244                 vec = self.get_vector(word)
245                 if vec is not None:
246                     norm1 = np.linalg.norm(analogy_vec)
247                     norm2 = np.linalg.norm(vec)
248                     if norm1 > 0 and norm2 > 0:
249                         sim = np.dot(analogy_vec, vec) / (norm1 * norm2)
250                         similarities.append((word, sim))
251
252         similarities.sort(key=lambda x: x[1], reverse=True)
253         return similarities[:topn]
254     except:
255         return []
256
257 def plot_loss_curve(self):
258     """绘制损失曲线"""
259     if not self.loss_history:
260         print("没有训练历史记录")
261         return
262
263     plt.figure(figsize=(10, 6))
264     plt.plot(self.loss_history)
265     plt.title('GloVe训练损失曲线')
266     plt.xlabel('Epoch')
267     plt.ylabel('Loss')
268     plt.grid(True)
269     plt.show()
270

```

```

271 def analyze_cooccurrence_matrix(self):
272     """分析共现矩阵统计信息"""
273     if self.cooccurrence_matrix is None:
274         print("共现矩阵未构建")
275         return
276
277     # 转换为密集矩阵进行分析 (小规模数据)
278     if self.vocab_size <= 1000:
279         dense_matrix = self.cooccurrence_matrix.toarray()
280
281         print("共现矩阵统计:")
282         print(f"非零元素比例: {self.cooccurrence_matrix.nnz / (self.vocab_size
283             **2):.4f}")
284         print(f"平均共现次数: {dense_matrix[dense_matrix > 0].mean():.2f}")
285         print(f"最大共现次数: {dense_matrix.max():.0f}")
286         print(f"共现次数标准差: {dense_matrix[dense_matrix > 0].std():.2f}")
287
288         # 分析词频分布
289         word_frequencies = np.array(dense_matrix.sum(axis=1)).flatten()
290         print(f"词频分布:")
291         print(f"    平均词频: {word_frequencies.mean():.2f}")
292         print(f"    词频标准差: {word_frequencies.std():.2f}")
293         print(f"    最高频词: {self.idx2word[word_frequencies.argmax():.2f}")
294         print(f"    最低频词: {self.idx2word[word_frequencies.argmin():.2f}")
295     else:
296         print("词汇表过大, 跳过详细分析")
297
298 class GloVeVisualizer:
299     """GloVe可视化工具"""
300
301     def __init__(self, model):
302         self.model = model
303
304     def plot_2d_embeddings(self, words=None, method='pca'):
305         """绘制2D词向量可视化"""
306         try:
307             from sklearn.decomposition import PCA
308             from sklearn.manifold import TSNE
309         except ImportError:
310             print("需要安装scikit-learn: pip install scikit-learn")
311             return
312
313         if words is None:
314             # 选择频率最高的词汇
315             words = list(self.model.word2idx.keys())[:20]
316
317         # 获取词向量
318         vectors = []
319         valid_words = []

```

```

319
320     for word in words:
321         vec = self.model.get_vector(word)
322         if vec is not None:
323             vectors.append(vec)
324             valid_words.append(word)
325
326     if len(vectors) < 2:
327         print("可用词向量不足")
328         return
329
330     vectors = np.array(vectors)
331
332     # 降维
333     if method == 'pca':
334         reducer = PCA(n_components=2)
335         vectors_2d = reducer.fit_transform(vectors)
336         title = f'GloVe词向量PCA可视化 (解释方差: {reducer.
337                 explained_variance_ratio_.sum():.2f})'
338     elif method == 'tsne':
339         reducer = TSNE(n_components=2, random_state=42)
340         vectors_2d = reducer.fit_transform(vectors)
341         title = 'GloVe词向量t-SNE可视化'
342     else:
343         print("不支持的降维方法")
344         return
345
346     # 绘图
347     plt.figure(figsize=(12, 8))
348     plt.scatter(vectors_2d[:, 0], vectors_2d[:, 1], alpha=0.7)
349
350     for i, word in enumerate(valid_words):
351         plt.annotate(word, (vectors_2d[i, 0], vectors_2d[i, 1]),
352                      xytext=(5, 5), textcoords='offset points',
353                      fontsize=9, alpha=0.8)
354
355     plt.title(title)
356     plt.xlabel('维度1')
357     plt.ylabel('维度2')
358     plt.grid(True, alpha=0.3)
359     plt.tight_layout()
360     plt.show()
361
362     def plot_similarity_heatmap(self, words):
363         """绘制词汇相似性热力图"""
364         try:
365             import seaborn as sns
366         except ImportError:
367             print("需要安装seaborn: pip install seaborn")

```

```

367         return
368
369     # 计算相似性矩阵
370     n = len(words)
371     similarity_matrix = np.zeros((n, n))
372
373     for i, word1 in enumerate(words):
374         for j, word2 in enumerate(words):
375             similarity_matrix[i, j] = self.model.similarity(word1, word2)
376
377     # 绘制热力图
378     plt.figure(figsize=(10, 8))
379     sns.heatmap(similarity_matrix,
380                 xticklabels=words,
381                 yticklabels=words,
382                 annot=True,
383                 fmt='.2f',
384                 cmap='RdYlBu_r',
385                 center=0)
386     plt.title('词汇相似性热力图')
387     plt.tight_layout()
388     plt.show()
389
390 # GloVe 完整演示
391 def demonstrate_glove():
392     """演示 GloVe"""
393     print("=" * 60)
394     print("GloVe 完整演示")
395     print("=" * 60)
396
397     # 扩展的训练数据
398     sentences = [
399         ['自然', '语言', '处理', '是', '人工', '智能', '的', '重要', '分支'],
400         ['机器', '学习', '广泛', '应用', '于', '自然', '语言', '处理'],
401         ['词向量', '是', '自然', '语言', '处理', '的', '基础', '技术'],
402         ['深度', '学习', '推动', '了', '自然', '语言', '处理', '发展'],
403         ['GloVe', '算法', '学习', '全局', '词汇', '共现', '信息'],
404         ['词汇', '共现', '矩阵', '包含', '语料库', '统计', '信息'],
405         ['机器', '学习', '模型', '需要', '大量', '训练', '数据'],
406         ['人工', '智能', '技术', '不断', '进步', '和', '发展'],
407         ['神经', '网络', '是', '深度', '学习', '的', '核心'],
408         ['算法', '优化', '提高', '模型', '性能', '和', '效率'],
409         ['数据', '挖掘', '从', '大量', '数据', '中', '发现', '模式'],
410         ['计算机', '视觉', '处理', '图像', '和', '视频', '信息'],
411         ['语音', '识别', '将', '音频', '转换', '为', '文本'],
412         ['知识', '图谱', '表示', '实体', '和', '关系', '信息'],
413         ['推荐', '系统', '根据', '用户', '偏好', '推荐', '内容']
414     ]
415

```

```

416 # 创建和训练模型
417 print("\n1. 创建GloVe模型...")
418 model = GloVe(vector_size=50, x_max=50, alpha=0.75, learning_rate=0.01)
419 model.build_vocab(sentences)
420 model.build_cooccurrence_matrix(sentences, window_size=3)
421
422 print("\n2. 分析共现矩阵...")
423 model.analyze_cooccurrence_matrix()
424
425 print("\n3. 训练模型...")
426 model.train(epochs=100, verbose=True)
427
428 print("\n4. 测试词汇相似性...")
429 test_words = ['自然', '语言', '机器', '学习', '深度', '人工']
430 for word in test_words:
431     similar_words = model.most_similar(word, topn=3)
432     print(f"{word}: {similar_words}")
433
434 print("\n5. 测试词汇类比...")
435 analogies = [
436     ('自然', '语言', '机器', '学习'),
437     ('深度', '学习', '人工', '智能'),
438     ('词汇', '共现', '数据', '信息')
439 ]
440
441 for a, b, c, expected in analogies:
442     result = model.analogy(a, b, c, topn=3)
443     print(f"{a} - {b} + {c} = {result}")
444
445 print("\n6. 可视化分析...")
446
447 # 创建可视化器
448 visualizer = GloVeVisualizer(model)
449
450 # 分析特定词汇
451 analysis_words = ['自然', '语言', '处理', '机器', '学习', '深度', '人工', '智能']
452
453 # 绘制相似性热力图
454 print("绘制相似性热力图...")
455 try:
456     visualizer.plot_similarity_heatmap(analysis_words)
457 except Exception as e:
458     print(f"热力图绘制失败: {e}")
459
460 # 绘制2D可视化
461 print("绘制2D词向量可视化...")
462 try:
463     visualizer.plot_2d_embeddings(analysis_words, method='pca')

```



```

464     except Exception as e:
465         print(f"2D可视化失败: {e}")
466
467     # 绘制损失曲线
468     print("绘制训练损失曲线...")
469     try:
470         model.plot_loss_curve()
471     except Exception as e:
472         print(f"损失曲线绘制失败: {e}")
473
474     print("\n7. 模型性能分析...")
475
476     # 词向量质量分析
477     print("词向量统计:")
478     all_vectors = []
479     for word in model.word2idx:
480         vec = model.get_vector(word)
481         if vec is not None:
482             all_vectors.append(vec)
483
484     if all_vectors:
485         all_vectors = np.array(all_vectors)
486         print(f"词向量平均范数: {np.linalg.norm(all_vectors, axis=1).mean():.3f}")
487         print(f"词向量标准差: {all_vectors.std():.3f}")
488         print(f"词向量维度: {all_vectors.shape[1]}")
489
490     return model
491
492 if __name__ == "__main__":
493     glove_model = demonstrate_glove()

```

Listing 1: GloVe 算法完整实现

4.2 3.2 N-gram 语言模型 (15 分钟)

4.2.1 N-gram 模型原理

N-gram 模型基于马尔可夫假设，认为当前词只依赖于前面有限个词：

$$P(w_i|w_1, w_2, \dots, w_{i-1}) \approx P(w_i|w_{i-n+1}, \dots, w_{i-1}) \quad (21)$$

常见的 N-gram 模型：

- Unigram: $P(w_i)$
- Bigram: $P(w_i|w_{i-1})$
- Trigram: $P(w_i|w_{i-2}, w_{i-1})$

4.2.2 参数估计

使用最大似然估计：

$$P(w_i|w_{i-n+1}, \dots, w_{i-1}) = \frac{C(w_{i-n+1}, \dots, w_{i-1}, w_i)}{C(w_{i-n+1}, \dots, w_{i-1})} \quad (22)$$

4.3 3.3 神经网络语言模型 (25 分钟)

4.3.1 神经网络语言模型的优势

神经网络语言模型相比 N-gram 模型有以下优势：

表 7: 神经网络语言模型 vs N-gram 模型

特性	N-gram 模型	神经网络语言模型
参数共享	每个 n-gram 独立参数 参数数量: $O(V ^n)$	词向量参数共享 参数数量: $O(V \cdot d)$
泛化能力	难以泛化到未见 n-gram 硬匹配	基于相似性泛化 软匹配
上下文长度	固定 n-1 个词 无法处理长距离依赖	可变长度 (RNN) 理论上无限制
语义理解	基于符号匹配 无语义相似性	基于分布式表示 捕获语义关系
计算复杂度	查表操作: $O(1)$ 训练快速	矩阵运算: $O(d^2)$ 训练较慢

4.3.2 FastText 算法实现

```
1 import numpy as np
2 from collections import defaultdict, Counter
3 import re
4 import math
5 import random
6
7 class FastText:
8     def __init__(self, vector_size=100, window=5, min_count=5,
9                 min_n=3, max_n=6, negative=5, bucket=2000000):
10
11         """
12         FastText 模型
13
14         Args:
15             vector_size: 词向量维度
16             window: 上下文窗口大小
17             min_count: 最小词频
18             min_n: 最小子词长度
19             max_n: 最大子词长度
20             negative: 负采样数量
```

```

20         bucket: 子词哈希桶大小
21     """
22     self.vector_size = vector_size
23     self.window = window
24     self.min_count = min_count
25     self.min_n = min_n
26     self.max_n = max_n
27     self.negative = negative
28     self.bucket = bucket
29
30     # 词汇表
31     self.word2idx = {}
32     self.idx2word = {}
33     self.vocab_size = 0
34     self.word_counts = Counter()
35
36     # 子词表
37     self.subword2idx = {}
38     self.subword_counts = Counter()
39
40     # 向量矩阵
41     self.word_vectors = None # 词向量
42     self.subword_vectors = None # 子词向量
43     self.output_vectors = None # 输出向量
44
45     # 负采样表
46     self.table = None
47     self.table_size = int(1e8)
48
49     def _hash_subword(self, subword):
50         """子词哈希函数"""
51         hash_value = 0
52         for char in subword:
53             hash_value = hash_value * 31 + ord(char)
54         return hash_value % self.bucket
55
56     def _get_subwords(self, word):
57         """提取词汇的子词"""
58         # 添加边界符号
59         word_with_boundary = '<' + word + '>'
60         subwords = []
61
62         # 提取不同长度的n-gram
63         for n in range(self.min_n, min(len(word_with_boundary), self.max_n) + 1):
64             for i in range(len(word_with_boundary) - n + 1):
65                 subword = word_with_boundary[i:i+n]
66                 subwords.append(subword)
67
68         # 添加完整词汇 (不带边界符号)

```

```

69         subwords.append(word)
70
71     return subwords
72
73     def _get_subword_indices(self, word):
74         """获取词汇的子词索引"""
75         subwords = self._get_subwords(word)
76         indices = []
77
78         for subword in subwords:
79             if subword in self.subword2idx:
80                 indices.append(self.subword2idx[subword])
81             else:
82                 # 使用哈希映射未见过的子词
83                 hash_idx = self._hash_subword(subword) + self.vocab_size
84                 indices.append(hash_idx)
85
86         return indices
87
88     def build_vocab(self, sentences):
89         """构建词汇表和子词表"""
90         print("构建词汇表...")
91
92         # 统计词频
93         for sentence in sentences:
94             for word in sentence:
95                 self.word_counts[word] += 1
96
97         # 过滤低频词
98         vocab_words = [word for word, count in self.word_counts.items()
99                        if count >= self.min_count]
100
101         # 建立词汇索引
102         self.word2idx = {word: idx for idx, word in enumerate(vocab_words)}
103         self.idx2word = {idx: word for word, idx in self.word2idx.items()}
104         self.vocab_size = len(vocab_words)
105
106         print(f"词汇表大小: {self.vocab_size}")
107
108         # 收集所有子词
109         all_subwords = set()
110         for word in vocab_words:
111             subwords = self._get_subwords(word)
112             all_subwords.update(subwords[:-1]) # 排除完整词汇
113
114         # 统计子词频率
115         for subword in all_subwords:
116             self.subword_counts[subword] += self.word_counts[word]
117

```

```

118     # 建立子词索引 (仅为频繁子词)
119     frequent_subwords = [sw for sw, count in self.subword_counts.items()
120                           if count >= self.min_count]
121     self.subword2idx = {subword: idx + self.vocab_size
122                        for idx, subword in enumerate(frequent_subwords)}
123
124     total_params = self.vocab_size + len(frequent_subwords) + self.bucket
125     print(f"子词表大小: {len(frequent_subwords)}")
126     print(f"总参数规模: {total_params}")
127
128     # 初始化向量
129     scale = 1.0 / self.vector_size
130     self.word_vectors = np.random.uniform(
131         -scale, scale, (total_params, self.vector_size)
132     ).astype(np.float32)
133
134     self.output_vectors = np.random.uniform(
135         -scale, scale, (self.vocab_size, self.vector_size)
136     ).astype(np.float32)
137
138     # 构建负采样表
139     self._build_negative_sampling_table()
140
141     def _build_negative_sampling_table(self):
142         """构建负采样表"""
143         # 基于词频构建采样表
144         word_pow = np.array([self.word_counts[self.idx2word[i]]**0.75
145                              for i in range(self.vocab_size)])
146         word_pow_sum = word_pow.sum()
147         word_prob = word_pow / word_pow_sum
148
149         # 构建累积分布
150         self.table = np.random.choice(self.vocab_size,
151                                       size=self.table_size,
152                                       p=word_prob)
153
154     def _get_negative_samples(self, positive_idx, num_samples):
155         """获取负采样"""
156         negative_samples = []
157         while len(negative_samples) < num_samples:
158             sample = self.table[random.randint(0, self.table_size - 1)]
159             if sample != positive_idx:
160                 negative_samples.append(sample)
161         return negative_samples
162
163     def _sigmoid(self, x):
164         """Sigmoid函数 (数值稳定版本) """
165         x = np.clip(x, -6, 6)
166         return 1.0 / (1.0 + np.exp(-x))

```

```

167
168 def _train_skipgram(self, center_word_idx, context_word_idx, learning_rate):
169     """Skip-gram训练"""
170     # 获取中心词的子词索引
171     center_word = self.idx2word[center_word_idx]
172     subword_indices = self._get_subword_indices(center_word)
173
174     # 计算中心词向量 (子词向量平均)
175     center_vector = np.mean([self.word_vectors[idx] for idx in subword_indices],
176                             axis=0)
177
178     # 正样本训练
179     target_vector = self.output_vectors[context_word_idx]
180     score = np.dot(center_vector, target_vector)
181     pred = self._sigmoid(score)
182     gradient = (1 - pred) * learning_rate
183
184     # 更新输出向量
185     self.output_vectors[context_word_idx] += gradient * center_vector
186
187     # 计算子词梯度
188     subword_gradient = gradient * target_vector
189
190     # 负样本训练
191     negative_samples = self._get_negative_samples(context_word_idx, self.
192                                                    negative)
193     for neg_idx in negative_samples:
194         neg_vector = self.output_vectors[neg_idx]
195         score = np.dot(center_vector, neg_vector)
196         pred = self._sigmoid(score)
197         gradient = -pred * learning_rate
198
199         self.output_vectors[neg_idx] += gradient * center_vector
200         subword_gradient += gradient * neg_vector
201
202     # 更新子词向量
203     for idx in subword_indices:
204         if idx < len(self.word_vectors):
205             self.word_vectors[idx] += subword_gradient / len(subword_indices)
206
207 def train(self, sentences, epochs=5, learning_rate=0.025):
208     """训练FastText模型"""
209     print(f"开始训练FastText模型，共{epochs}轮...")
210
211     total_words = sum(len(sentence) for sentence in sentences)
212     words_processed = 0
213
214     for epoch in range(epochs):
215         print(f"第{epoch+1}轮训练...")

```

```

214         random.shuffle(sentences)
215
216     for sentence in sentences:
217         # 过滤词汇表中的词
218         sentence_indices = [self.word2idx[word] for word in sentence
219                             if word in self.word2idx]
220
221         if len(sentence_indices) < 2:
222             continue
223
224         # Skip-gram训练
225         for i, center_idx in enumerate(sentence_indices):
226             # 动态窗口大小
227             reduced_window = random.randint(1, self.window)
228             start = max(0, i - reduced_window)
229             end = min(len(sentence_indices), i + reduced_window + 1)
230
231             for j in range(start, end):
232                 if i != j:
233                     context_idx = sentence_indices[j]
234                     self._train_skipgram(center_idx, context_idx,
235                                           learning_rate)
236
237             words_processed += len(sentence_indices)
238
239         # 学习率衰减
240         if words_processed % 10000 == 0:
241             progress = words_processed / (total_words * epochs)
242             learning_rate = 0.025 * (1 - progress)
243             learning_rate = max(learning_rate, 0.0001)
244
245     print("训练完成!")
246
247     def get_word_vector(self, word):
248         """获取词向量 (通过子词平均) """
249         if word in self.word2idx:
250             # 已知词汇: 结合词向量和子词向量
251             word_idx = self.word2idx[word]
252             word_vec = self.word_vectors[word_idx]
253
254             # 子词向量
255             subword_indices = self._get_subword_indices(word)
256             subword_vecs = [self.word_vectors[idx] for idx in subword_indices[:-1]]
257             # 排除词本身
258
259             if subword_vecs:
260                 subword_avg = np.mean(subword_vecs, axis=0)
261                 return (word_vec + subword_avg) / 2
262             else:

```

```

261         return word_vec
262     else:
263         # 未知词汇：仅使用子词向量
264         subword_indices = self._get_subword_indices(word)
265         subword_vecs = [self.word_vectors[idx] for idx in subword_indices
266                         if idx < len(self.word_vectors)]
267
268         if subword_vecs:
269             return np.mean(subword_vecs, axis=0)
270         else:
271             return np.zeros(self.vector_size)
272
273     def similarity(self, word1, word2):
274         """计算词汇相似度"""
275         vec1 = self.get_word_vector(word1)
276         vec2 = self.get_word_vector(word2)
277
278         # 余弦相似度
279         norm1 = np.linalg.norm(vec1)
280         norm2 = np.linalg.norm(vec2)
281
282         if norm1 == 0 or norm2 == 0:
283             return 0
284
285         return np.dot(vec1, vec2) / (norm1 * norm2)
286
287     def most_similar(self, word, topn=10):
288         """找到最相似的词汇"""
289         word_vec = self.get_word_vector(word)
290         similarities = []
291
292         # 与词汇表中的词比较
293         for other_word in self.word2idx:
294             if other_word != word:
295                 sim = self.similarity(word, other_word)
296                 similarities.append((other_word, sim))
297
298         similarities.sort(key=lambda x: x[1], reverse=True)
299         return similarities[:topn]
300
301     def get_oov_handling_demo(self, test_words):
302         """演示OOV词汇处理"""
303         print("OOV词汇处理演示:")
304         for word in test_words:
305             is_known = word in self.word2idx
306             vec = self.get_word_vector(word)
307             vec_norm = np.linalg.norm(vec)
308
309             # 分析子词构成

```



```

310         subwords = self._get_subwords(word)
311         known_subwords = [sw for sw in subwords[:-1] if sw in self.subword2idx]
312
313         print(f"\n词汇: {word}")
314         print(f" 是否已知: {is_known}")
315         print(f" 向量范数: {vec_norm:.3f}")
316         print(f" 子词数量: {len(subwords)-1}")
317         print(f" 已知子词: {len(known_subwords)}/{len(subwords)-1}")
318
319         if known_subwords:
320             print(f" 主要子词: {known_subwords[:5]}")
321
322     def analyze_morphology(self, word_pairs):
323         """分析形态学关系"""
324         print("形态学关系分析:")
325         for word1, word2 in word_pairs:
326             sim = self.similarity(word1, word2)
327
328             # 计算共同子词
329             subwords1 = set(self._get_subwords(word1)[:-1])
330             subwords2 = set(self._get_subwords(word2)[:-1])
331             common_subwords = subwords1.intersection(subwords2)
332
333             print(f"\n{word1} - {word2}:")
334             print(f" 相似度: {sim:.3f}")
335             print(f" 共同子词数: {len(common_subwords)}")
336             if common_subwords:
337                 print(f" 共同子词: {list(common_subwords)[:5]}")
338
339     class FastTextEvaluator:
340         """FastText评估器"""
341
342         def __init__(self, model):
343             self.model = model
344
345         def evaluate_oov_capability(self, known_words, unknown_words):
346             """评估OOV处理能力"""
347             results = {
348                 'known_word_similarities': [],
349                 'unknown_word_qualities': [],
350                 'morphological_consistency': []
351             }
352
353             # 测试已知词汇的相似性
354             for i, word1 in enumerate(known_words):
355                 for word2 in known_words[i+1:]:
356                     sim = self.model.similarity(word1, word2)
357                     results['known_word_similarities'].append(sim)
358

```

```

359     # 测试未知词汇的向量质量
360     for word in unknown_words:
361         vec = self.model.get_word_vector(word)
362         quality = np.linalg.norm(vec) # 使用向量范数作为质量指标
363         results['unknown_word_qualities'].append(quality)
364
365     # 测试形态学一致性 (如果提供相关词对)
366     morphological_pairs = [
367         ('学习', '学会'), # 相关概念
368         ('机器', '机械'), # 相似词根
369         ('智能', '智慧'), # 语义相关
370     ]
371
372     for word1, word2 in morphological_pairs:
373         sim = self.model.similarity(word1, word2)
374         results['morphological_consistency'].append(sim)
375
376     return results
377
378     def compare_with_word2vec_simulation(self, test_words):
379         """模拟与Word2Vec的比较"""
380         print("FastText vs Word2Vec (模拟) 比较:")
381
382         for word in test_words:
383             is_known = word in self.model.word2idx
384             fasttext_vec = self.model.get_word_vector(word)
385             fasttext_norm = np.linalg.norm(fasttext_vec)
386
387             print(f"\n词汇: {word}")
388             print(f"    Word2Vec处理: {'有向量' if is_known else '无法处理(OOV)'}")
389             print(f"    FastText处理: 有向量 (norm: {fasttext_norm:.3f})")
390
391             if is_known:
392                 # 找相似词
393                 similar = self.model.most_similar(word, topn=3)
394                 print(f"    相似词: {[w for w, s in similar]}")
395
396     # FastText完整演示
397     def demonstrate_fasttext():
398         """演示FastText"""
399         print("=" * 60)
400         print("FastText完整演示")
401         print("=" * 60)
402
403     # 扩展训练数据 (包含形态变化)
404     sentences = [
405         ['学习', '机器', '学习', '算法'],
406         ['学会', '深度', '学习', '技术'],
407         ['学者', '研究', '人工', '智能'],

```

```

408     ['教学', '自然', '语言', '处理'],
409     ['同学', '一起', '学习', '编程'],
410     ['数学', '基础', '很', '重要'],
411     ['算法', '优化', '提升', '性能'],
412     ['模型', '训练', '需要', '数据'],
413     ['智能', '系统', '应用', '广泛'],
414     ['智慧', '城市', '建设', '发展'],
415     ['机器', '人', '协作', '未来'],
416     ['机械', '工程', '自动化', '技术'],
417     ['计算', '机', '科学', '进步'],
418     ['计算器', '帮助', '数学', '运算'],
419     ['处理', '大量', '数据', '信息'],
420     ['处理器', '性能', '不断', '提升']
421 ]
422
423 print("\n1. 创建FastText模型...")
424 model = FastText(vector_size=50, min_n=2, max_n=4, min_count=1, window=3)
425 model.build_vocab(sentences)
426
427 print("\n2. 训练模型...")
428 model.train(sentences, epochs=50)
429
430 print("\n3. 测试已知词汇相似性...")
431 known_words = ['学习', '机器', '智能', '算法', '数据']
432 for word in known_words:
433     similar = model.most_similar(word, topn=3)
434     print(f"{word}: {similar}")
435
436 print("\n4. 测试OOV词汇处理...")
437 oov_words = ['学生', '教师', '算术', '智能化', '机器人']
438 model.get_oov_handling_demo(oov_words)
439
440 print("\n5. 形态学关系分析...")
441 morphological_pairs = [
442     ('学习', '学会'),
443     ('学习', '学者'),
444     ('机器', '机械'),
445     ('智能', '智慧'),
446     ('计算', '计算器'),
447     ('处理', '处理器')
448 ]
449 model.analyze_morphology(morphological_pairs)
450
451 print("\n6. 评估模型性能...")
452 evaluator = FastTextEvaluator(model)
453
454 # OOV能力评估
455 oov_results = evaluator.evaluate_oov_capability(known_words, oov_words)
456

```

```

457 print("OOV 处理能力评估:")
458 print(f" 已知词平均相似度: {np.mean(oov_results['known_word_similarities']):.3f}
      f}")
459 print(f" 未知词平均向量质量: {np.mean(oov_results['unknown_word_qualities']):.3f}
      f}")
460 print(f" 形态学一致性: {np.mean(oov_results['morphological_consistency']):.3f}"
      )
461
462 # 与 Word2Vec 比较
463 print("\n7. 与 Word2Vec 对比...")
464 all_test_words = known_words + oov_words
465 evaluator.compare_with_word2vec_simulation(all_test_words)
466
467 return model
468
469 if __name__ == "__main__":
470     fasttext_model = demonstrate_fasttext()

```

Listing 2: FastText 完整实现

4.3.3 前馈神经网络语言模型

Bengio 等人在 2003 年提出的神经概率语言模型是第一个成功的神经网络语言模型。

模型架构:

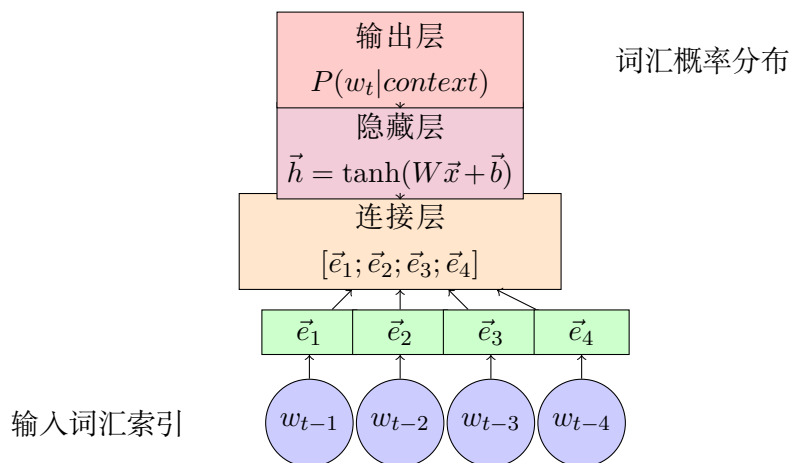


图 9: 前馈神经网络语言模型结构

数学表述:

1. 词嵌入层:

$$\vec{e}_i = E \cdot \text{onehot}(w_{t-i}) \quad (23)$$

其中 $E \in \mathbb{R}^{d \times |V|}$ 是词嵌入矩阵, d 是嵌入维度。

2. 连接层:

$$\vec{x} = [\vec{e}_1; \vec{e}_2; \dots; \vec{e}_{n-1}] \quad (24)$$

其中 $[\cdot; \cdot]$ 表示向量连接操作。

3. 隐藏层：

$$\vec{h} = \tanh(W_h \vec{x} + \vec{b}_h) \quad (25)$$

其中 $W_h \in \mathbb{R}^{H \times (n-1)d}$, H 是隐藏层大小。

4. 输出层：

$$\vec{o} = W_o \vec{h} + \vec{b}_o \quad (26)$$

$$P(w_t | w_{t-n+1}, \dots, w_{t-1}) = \frac{\exp(o_{w_t})}{\sum_{w=1}^{|V|} \exp(o_w)} \quad (27)$$

损失函数：

$$L = - \sum_{i=1}^N \log P(w_i | w_{i-n+1}, \dots, w_{i-1}) \quad (28)$$

4.3.4 循环神经网络语言模型

RNN 语言模型能够处理任意长度的序列，是神经网络语言模型的重要发展。

RNN 语言模型架构：

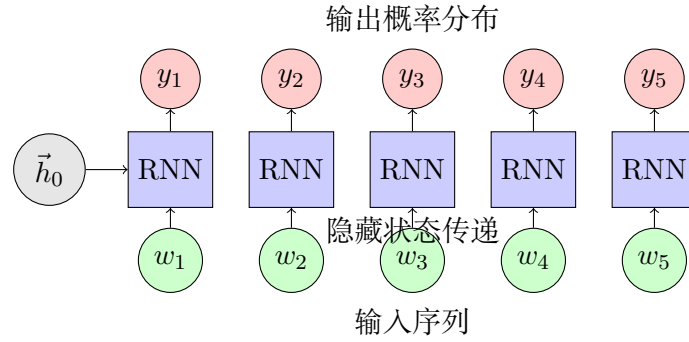


图 10: RNN 语言模型结构

RNN 更新公式：

$$\vec{h}_t = \tanh(W_{hh} \vec{h}_{t-1} + W_{xh} \vec{e}_t + \vec{b}_h) \quad (29)$$

$$\vec{o}_t = W_{ho} \vec{h}_t + \vec{b}_o \quad (30)$$

$$P(w_t | w_1, \dots, w_{t-1}) = \text{softmax}(\vec{o}_t) \quad (31)$$

其中：- $\vec{e}_t$ 是词 w_t 的嵌入向量 - $\vec{h}_t$ 是时刻 t 的隐藏状态 - W_{hh}, W_{xh}, W_{ho} 是权重矩阵

LSTM 语言模型：

为了解决 RNN 的梯度消失问题，LSTM 语言模型引入了门控机制：

$$\vec{f}_t = \sigma(W_f \cdot [\vec{h}_{t-1}, \vec{x}_t] + \vec{b}_f) \quad (\text{遗忘门}) \quad (32)$$

$$\vec{i}_t = \sigma(W_i \cdot [\vec{h}_{t-1}, \vec{x}_t] + \vec{b}_i) \quad (\text{输入门}) \quad (33)$$

$$\tilde{\vec{C}}_t = \tanh(W_C \cdot [\vec{h}_{t-1}, \vec{x}_t] + \vec{b}_C) \quad (\text{候选值}) \quad (34)$$

$$\vec{C}_t = \vec{f}_t * \vec{C}_{t-1} + \vec{i}_t * \tilde{\vec{C}}_t \quad (\text{细胞状态}) \quad (35)$$

$$\vec{o}_t = \sigma(W_o \cdot [\vec{h}_{t-1}, \vec{x}_t] + \vec{b}_o) \quad (\text{输出门}) \quad (36)$$

$$\vec{h}_t = \vec{o}_t * \tanh(\vec{C}_t) \quad (\text{隐藏状态}) \quad (37)$$

4.3.5 神经网络语言模型实现

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import torch.nn.functional as F
5 from torch.utils.data import Dataset, DataLoader
6 import numpy as np
7 import math
8 import random
9 from collections import Counter, defaultdict
10
11 class LanguageModelDataset(Dataset):
12     """语言模型数据集"""
13
14     def __init__(self, sentences, seq_length, word2idx, mode='rnn'):
15         """
16         Args:
17             sentences: 句子列表
18             seq_length: 序列长度
19             word2idx: 词汇到索引的映射
20             mode: 'rnn' 或 'feedforward'
21         """
22         self.seq_length = seq_length
23         self.word2idx = word2idx
24         self.mode = mode
25
26         # 处理数据
27         self.data = []
28         for sentence in sentences:
29             # 添加开始和结束标记
30             tokens = ['<s>'] + sentence + ['</s>']
31             indices = [word2idx.get(token, word2idx['<unk>']) for token in tokens]
32             self.data.extend(indices)
33
34     def __len__(self):
35         return len(self.data) - self.seq_length
36

```

```

37     def __getitem__(self, idx):
38         if self.mode == 'rnn':
39             # RNN模式：输入序列和目标序列
40             input_seq = torch.tensor(self.data[idx:idx + self.seq_length], dtype=
41                                     torch.long)
42             target_seq = torch.tensor(self.data[idx + 1:idx + self.seq_length + 1],
43                                     dtype=torch.long)
44             return input_seq, target_seq
45         else:
46             # 前馈模式：固定上下文和目标词
47             context = torch.tensor(self.data[idx:idx + self.seq_length], dtype=torch
48                                   .long)
49             target = torch.tensor(self.data[idx + self.seq_length], dtype=torch.long
50                                   )
51             return context, target
52
53 class FeedforwardLanguageModel(nn.Module):
54     """前馈神经网络语言模型"""
55
56     def __init__(self, vocab_size, embedding_dim=128, hidden_dim=256, context_size
57                 =4):
58         super(FeedforwardLanguageModel, self).__init__()
59
60         self.vocab_size = vocab_size
61         self.embedding_dim = embedding_dim
62         self.hidden_dim = hidden_dim
63         self.context_size = context_size
64
65         # 模型层
66         self.embedding = nn.Embedding(vocab_size, embedding_dim)
67         self.fc1 = nn.Linear(context_size * embedding_dim, hidden_dim)
68         self.fc2 = nn.Linear(hidden_dim, hidden_dim)
69         self.output = nn.Linear(hidden_dim, vocab_size)
70         self.dropout = nn.Dropout(0.2)
71
72         # 初始化权重
73         self._init_weights()
74
75     def _init_weights(self):
76         """初始化权重"""
77         nn.init.uniform_(self.embedding.weight, -0.1, 0.1)
78         nn.init.xavier_uniform_(self.fc1.weight)
79         nn.init.xavier_uniform_(self.fc2.weight)
80         nn.init.xavier_uniform_(self.output.weight)
81
82     def forward(self, context):
83         """前向传播"""
84         # 词嵌入
85         embedded = self.embedding(context) # (batch_size, context_size,

```

```

        embedding_dim)
81
82 # 展平连接
83 flattened = embedded.view(embedded.size(0), -1) # (batch_size, context_size
        * embedding_dim)
84
85 # 前馈网络
86 hidden1 = torch.tanh(self.fc1(flattened))
87 hidden1 = self.dropout(hidden1)
88
89 hidden2 = torch.tanh(self.fc2(hidden1))
90 hidden2 = self.dropout(hidden2)
91
92 # 输出层
93 output = self.output(hidden2) # (batch_size, vocab_size)
94
95 return output
96
97 class RNNLanguageModel(nn.Module):
98     """RNN 语言模型"""
99
100     def __init__(self, vocab_size, embedding_dim=128, hidden_dim=256, num_layers=2,
101                 rnn_type='LSTM'):
102         super(RNNLanguageModel, self).__init__()
103
104         self.vocab_size = vocab_size
105         self.embedding_dim = embedding_dim
106         self.hidden_dim = hidden_dim
107         self.num_layers = num_layers
108         self.rnn_type = rnn_type
109
110         # 模型层
111         self.embedding = nn.Embedding(vocab_size, embedding_dim)
112
113         if rnn_type == 'LSTM':
114             self.rnn = nn.LSTM(embedding_dim, hidden_dim, num_layers,
115                               dropout=0.2, batch_first=True)
116         elif rnn_type == 'GRU':
117             self.rnn = nn.GRU(embedding_dim, hidden_dim, num_layers,
118                               dropout=0.2, batch_first=True)
119         else:
120             self.rnn = nn.RNN(embedding_dim, hidden_dim, num_layers,
121                               dropout=0.2, batch_first=True, nonlinearity='tanh')
122
123         self.output = nn.Linear(hidden_dim, vocab_size)
124         self.dropout = nn.Dropout(0.2)
125
126         # 初始化权重
127         self._init_weights()

```



```

127
128 def _init_weights(self):
129     """初始化权重"""
130     nn.init.uniform_(self.embedding.weight, -0.1, 0.1)
131
132     # 初始化RNN权重
133     for name, param in self.rnn.named_parameters():
134         if 'weight' in name:
135             nn.init.orthogonal_(param)
136         elif 'bias' in name:
137             nn.init.zeros_(param)
138
139     nn.init.xavier_uniform_(self.output.weight)
140
141 def forward(self, x, hidden=None):
142     """前向传播"""
143     batch_size = x.size(0)
144
145     # 词嵌入
146     embedded = self.embedding(x) # (batch_size, seq_len, embedding_dim)
147     embedded = self.dropout(embedded)
148
149     # RNN
150     if hidden is None:
151         rnn_out, hidden = self.rnn(embedded)
152     else:
153         rnn_out, hidden = self.rnn(embedded, hidden)
154
155     # 输出投影
156     output = self.output(rnn_out) # (batch_size, seq_len, vocab_size)
157
158     return output, hidden
159
160 def init_hidden(self, batch_size, device):
161     """初始化隐藏状态"""
162     if self.rnn_type == 'LSTM':
163         h0 = torch.zeros(self.num_layers, batch_size, self.hidden_dim).to(device)
164         c0 = torch.zeros(self.num_layers, batch_size, self.hidden_dim).to(device)
165         return (h0, c0)
166     else:
167         h0 = torch.zeros(self.num_layers, batch_size, self.hidden_dim).to(device)
168         return h0
169
170 class LanguageModelTrainer:
171     """语言模型训练器"""
172

```

```

173     def __init__(self, model, device='cpu'):
174         self.model = model
175         self.device = device
176         self.model.to(device)
177
178         # 训练历史
179         self.train_losses = []
180         self.val_losses = []
181         self.perplexities = []
182
183     def train_epoch(self, dataloader, optimizer, criterion, clip_grad=1.0):
184         """训练一个epoch"""
185         self.model.train()
186         total_loss = 0
187         total_tokens = 0
188
189         for batch_idx, (inputs, targets) in enumerate(dataloader):
190             inputs = inputs.to(self.device)
191             targets = targets.to(self.device)
192
193             optimizer.zero_grad()
194
195             if isinstance(self.model, RNNLanguageModel):
196                 # RNN模型
197                 outputs, _ = self.model(inputs)
198                 outputs = outputs.view(-1, self.model.vocab_size)
199                 targets = targets.view(-1)
200             else:
201                 # 前馈模型
202                 outputs = self.model(inputs)
203
204             loss = criterion(outputs, targets)
205             loss.backward()
206
207             # 梯度裁剪
208             if clip_grad > 0:
209                 torch.nn.utils.clip_grad_norm_(self.model.parameters(), clip_grad)
210
211             optimizer.step()
212
213             total_loss += loss.item()
214             total_tokens += targets.numel()
215
216         avg_loss = total_loss / len(dataloader)
217         return avg_loss
218
219     def evaluate(self, dataloader, criterion):
220         """评估模型"""
221         self.model.eval()

```

```

222     total_loss = 0
223     total_tokens = 0
224
225     with torch.no_grad():
226         for inputs, targets in dataloader:
227             inputs = inputs.to(self.device)
228             targets = targets.to(self.device)
229
230             if isinstance(self.model, RNNLanguageModel):
231                 outputs, _ = self.model(inputs)
232                 outputs = outputs.view(-1, self.model.vocab_size)
233                 targets = targets.view(-1)
234             else:
235                 outputs = self.model(inputs)
236
237             loss = criterion(outputs, targets)
238             total_loss += loss.item()
239             total_tokens += targets.numel()
240
241     avg_loss = total_loss / len(dataloader)
242     perplexity = math.exp(avg_loss)
243     return avg_loss, perplexity
244
245 def train(self, train_loader, val_loader, num_epochs, learning_rate=0.001):
246     """训练模型"""
247     criterion = nn.CrossEntropyLoss()
248     optimizer = optim.Adam(self.model.parameters(), lr=learning_rate)
249     scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=10, gamma=0.1)
250
251     print(f"开始训练 {num_epochs} 个epoch...")
252
253     for epoch in range(num_epochs):
254         # 训练
255         train_loss = self.train_epoch(train_loader, optimizer, criterion)
256
257         # 验证
258         val_loss, perplexity = self.evaluate(val_loader, criterion)
259
260         # 更新学习率
261         scheduler.step()
262
263         # 记录历史
264         self.train_losses.append(train_loss)
265         self.val_losses.append(val_loss)
266         self.perplexities.append(perplexity)
267
268         if epoch % 5 == 0:
269             print(f"Epoch {epoch:3d}: Train Loss = {train_loss:.4f}, "
270                   f"Val Loss = {val_loss:.4f}, Perplexity = {perplexity:.2f}")

```

```

271
272     print("训练完成!")
273
274     def generate_text(self, word2idx, idx2word, start_words=None, max_length=20,
275                       temperature=1.0):
276         """生成文本"""
277         self.model.eval()
278
279         if start_words is None:
280             start_words = ['<s>']
281
282         # 转换为索引
283         input_ids = [word2idx.get(word, word2idx['<unk>']) for word in start_words]
284         generated = list(input_ids)
285
286         with torch.no_grad():
287             if isinstance(self.model, RNNLanguageModel):
288                 # RNN生成
289                 hidden = self.model.init_hidden(1, self.device)
290
291                 # 处理初始输入
292                 for i in range(len(input_ids) - 1):
293                     input_tensor = torch.tensor([[input_ids[i]]], device=self.device)
294                     _, hidden = self.model(input_tensor, hidden)
295
296                 # 生成后续词汇
297                 input_tensor = torch.tensor([[input_ids[-1]]], device=self.device)
298
299                 for _ in range(max_length):
300                     output, hidden = self.model(input_tensor, hidden)
301
302                     # 应用温度
303                     logits = output[0, -1] / temperature
304                     probabilities = F.softmax(logits, dim=0)
305
306                     # 采样
307                     next_word_id = torch.multinomial(probabilities, 1).item()
308
309                     if next_word_id == word2idx.get('</s>', 0):
310                         break
311
312                     generated.append(next_word_id)
313                     input_tensor = torch.tensor([[next_word_id]], device=self.device)
314
315             else:
316                 # 前馈模型生成
317                 context_size = self.model.context_size

```

```

317
318     # 确保有足够的上下文
319     while len(generated) < context_size:
320         generated.insert(0, word2idx.get('<s>', 0))
321
322     for _ in range(max_length):
323         # 取最后context_size个词作为输入
324         context = torch.tensor([generated[-context_size:]], device=self.
325                                 device)
326
327         output = self.model(context)
328
329         # 应用温度
330         logits = output[0] / temperature
331         probabilities = F.softmax(logits, dim=0)
332
333         # 采样
334         next_word_id = torch.multinomial(probabilities, 1).item()
335
336         if next_word_id == word2idx.get('</s>', 0):
337             break
338
339         generated.append(next_word_id)
340
341     # 转换回词汇
342     generated_words = []
343     for idx in generated:
344         word = idx2word.get(idx, '<unk>')
345         if word not in [<s>', '</s>']:
346             generated_words.append(word)
347
348     return generated_words
349
350 class LanguageModelComparator:
351     """语言模型比较器"""
352
353     def __init__(self):
354         self.models = {}
355         self.trainers = {}
356
357     def add_model(self, name, model, trainer):
358         """添加模型"""
359         self.models[name] = model
360         self.trainers[name] = trainer
361
362     def compare_perplexity(self, test_loader):
363         """比较困惑度"""
364         results = {}
365         criterion = nn.CrossEntropyLoss()

```

```

365
366     print("困惑度比较:")
367     print("-" * 40)
368
369     for name, trainer in self.trainers.items():
370         _, perplexity = trainer.evaluate(test_loader, criterion)
371         results[name] = perplexity
372         print(f"{name:20}: {perplexity:.2f}")
373
374     return results
375
376 def compare_generation(self, word2idx, idx2word, prompts, num_samples=3):
377     """比较生成质量"""
378     print("\n文本生成比较:")
379     print("-" * 40)
380
381     for prompt in prompts:
382         print(f"\n提示: {' '.join(prompt)}")
383
384         for name, trainer in self.trainers.items():
385             print(f"\n{name}:")
386
387             for i in range(num_samples):
388                 generated = trainer.generate_text(
389                     word2idx, idx2word, prompt, max_length=10
390                 )
391                 print(f"    {i+1}. {' '.join(generated)}")
392
393 # 完整演示
394 def demonstrate_neural_language_models():
395     """演示神经网络语言模型"""
396     print("=" * 80)
397     print("神经网络语言模型完整演示")
398     print("=" * 80)
399
400 # 准备数据
401 sentences = [
402     ['我', '喜欢', '机器', '学习'],
403     ['机器', '学习', '很', '有趣'],
404     ['深度', '学习', '是', '机器', '学习', '的', '分支'],
405     ['自然', '语言', '处理', '使用', '机器', '学习'],
406     ['神经', '网络', '是', '深度', '学习', '的', '基础'],
407     ['人工', '智能', '包含', '机器', '学习'],
408     ['算法', '优化', '很', '重要'],
409     ['数据', '是', '机器', '学习', '的', '燃料'],
410     ['模型', '训练', '需要', '大量', '数据'],
411     ['深度', '学习', '模型', '性能', '优秀'],
412     ['自然', '语言', '理解', '技术', '进步'],
413     ['人工', '智能', '应用', '广泛'],

```

```

414         ['机器', '学习', '算法', '不断', '改进'],
415         ['深度', '学习', '推动', '技术', '发展'],
416         ['我', '学习', '人工', '智能', '技术']
417     ]
418
419     print(f"训练数据: {len(sentences)} 个句子")
420
421     # 构建词汇表
422     vocab = set()
423     for sentence in sentences:
424         vocab.update(sentence)
425
426     # 添加特殊符号
427     vocab.update(['<s>', '</s>', '<unk>'])
428
429     word2idx = {word: idx for idx, word in enumerate(sorted(vocab))}
430     idx2word = {idx: word for word, idx in word2idx.items()}
431     vocab_size = len(vocab)
432
433     print(f"词汇表大小: {vocab_size}")
434
435     # 准备数据集
436     train_sentences = sentences[:12]
437     val_sentences = sentences[12:]
438
439     # 创建比较器
440     comparator = LanguageModelComparator()
441     device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
442     print(f"使用设备: {device}")
443
444     # 1. 前馈神经网络语言模型
445     print("\n1. 训练前馈神经网络语言模型...")
446
447     ffnn_dataset = LanguageModelDataset(train_sentences, seq_length=3,
448                                         word2idx=word2idx, mode='feedforward')
449     ffnn_val_dataset = LanguageModelDataset(val_sentences, seq_length=3,
450                                             word2idx=word2idx, mode='feedforward')
451
452     ffnn_loader = DataLoader(ffnn_dataset, batch_size=16, shuffle=True)
453     ffnn_val_loader = DataLoader(ffnn_val_dataset, batch_size=16, shuffle=False)
454
455     ffnn_model = FeedforwardLanguageModel(vocab_size, embedding_dim=32,
456                                           hidden_dim=64, context_size=3)
457     ffnn_trainer = LanguageModelTrainer(ffnn_model, device)
458     ffnn_trainer.train(ffnn_loader, ffnn_val_loader, num_epochs=50, learning_rate
459                       =0.01)
460
461     comparator.add_model('前馈神经网络', ffnn_model, ffnn_trainer)

```

```

462 # 2. RNN语言模型
463 print("\n2. 训练RNN语言模型...")
464
465 rnn_dataset = LanguageModelDataset(train_sentences, seq_length=8,
466                                     word2idx=word2idx, mode='rnn')
467 rnn_val_dataset = LanguageModelDataset(val_sentences, seq_length=8,
468                                         word2idx=word2idx, mode='rnn')
469
470 rnn_loader = DataLoader(rnn_dataset, batch_size=8, shuffle=True)
471 rnn_val_loader = DataLoader(rnn_val_dataset, batch_size=8, shuffle=False)
472
473 rnn_model = RNNLanguageModel(vocab_size, embedding_dim=32,
474                               hidden_dim=64, num_layers=2, rnn_type='RNN')
475 rnn_trainer = LanguageModelTrainer(rnn_model, device)
476 rnn_trainer.train(rnn_loader, rnn_val_loader, num_epochs=50, learning_rate=0.01)
477
478 comparator.add_model('RNN', rnn_model, rnn_trainer)
479
480 # 3. LSTM语言模型
481 print("\n3. 训练LSTM语言模型...")
482
483 lstm_model = RNNLanguageModel(vocab_size, embedding_dim=32,
484                                hidden_dim=64, num_layers=2, rnn_type='LSTM')
485 lstm_trainer = LanguageModelTrainer(lstm_model, device)
486 lstm_trainer.train(rnn_loader, rnn_val_loader, num_epochs=50, learning_rate
487                   =0.01)
488
489 comparator.add_model('LSTM', lstm_model, lstm_trainer)
490
491 # 4. GRU语言模型
492 print("\n4. 训练GRU语言模型...")
493
494 gru_model = RNNLanguageModel(vocab_size, embedding_dim=32,
495                               hidden_dim=64, num_layers=2, rnn_type='GRU')
496
497 gru_LSTM & Penn Treebank & 60-80 \\
498 Transformer & Penn Treebank & 50-70 \\
499 GPT-2 & WebText & 35-45 \\
500 GPT-3 & 大规模语料 & 20-30 \\
501 \hline
502 3-gram & 中文语料 & 200-350 \\
503 LSTM & 中文语料 & 80-120 \\
504 BERT & 中文语料 & 40-60 \\
505 \hline
506 \end{tabular}
507 \end{table}
508
509 \subsection{3.2 N-gram语言模型 (15分钟)}
510
511 \subsubsection{N-gram模型原理深入}

```



```

510
511 N-gram模型基于马尔可夫假设，认为当前词只依赖于前面有限个词：
512
513 \begin{equation}
514 P(w_i | w_1, w_2, \dots, w_{i-1}) \approx P(w_i | w_{i-n+1}, \dots, w_{i-1})
515 \end{equation}
516
517 \textbf{马尔可夫假设的合理性：}
518 \begin{itemize}
519     \item \textbf{计算可行性}：避免了指数级参数空间
520     \item \textbf{统计充分性}：有限上下文包含大部分预测信息
521     \item \textbf{语言学依据}：大多数语法依赖是局部的
522 \end{itemize}
523
524 \textbf{常见N-gram模型的数学形式：}
525
526 \textbf{1. Unigram模型：}
527 \begin{equation}
528 P(w_i) = \frac{C(w_i)}{N}
529 \end{equation}
530 其中  $N$  是总词数， $C(w_i)$  是词  $w_i$  的出现次数。
531
532 \textbf{2. Bigram模型：}
533 \begin{equation}
534 P(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}
535 \end{equation}
536
537 \textbf{3. Trigram模型：}
538 \begin{equation}
539 P(w_i | w_{i-2}, w_{i-1}) = \frac{C(w_{i-2}, w_{i-1}, w_i)}{C(w_{i-2}, w_{i-1})}
540 \end{equation}
541
542 \subsubsection{平滑技术详解}
543
544 N-gram模型面临的核心问题是数据稀疏性。即使在大型语料库中，许多可能的n-gram组合也从未
    出现过，导致零概率问题。
545
546 \textbf{1. 拉普拉斯平滑 (Add-one Smoothing)}
547
548 最简单的平滑方法：
549 \begin{equation}
550 P_{\text{smooth}}(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + V}
551 \end{equation}
552
553 其中  $V$  是词汇表大小。
554
555 \textbf{问题}：给未见n-gram分配过多概率质量。
556
557 \textbf{2. Add- 平滑}

```

```

558
559 改进版本，使用小于1的：
560 \begin{equation}
561 P_{\text{smooth}}(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i) + \alpha}{C(w_{i-1}) + \alpha V}
562 \end{equation}
563
564 通常  $\alpha = 0.01$  或更小。
565
566 \textbf{3. Good-Turing平滑}
567
568 基于频率统计的原理：
569 \begin{equation}
570 C^* = (c + 1) \frac{N_{c+1}}{N_c}
571 \end{equation}
572
573 其中：
574 -  $c$  是原始计数
575 -  $N_c$  是出现  $c$  次的  $n$ -gram 数量
576 -  $C^*$  是调整后的计数
577
578 \textbf{4. Katz回退 (Katz Backoff)}
579
580 当高阶  $n$ -gram 不可用时，回退到低阶：
581 \begin{equation}
582 P_{\text{katz}}(w_i | w_{i-n+1}^{i-1}) = \begin{cases}
583 d \cdot P_{\text{ML}}(w_i | w_{i-n+1}^{i-1}) & \text{if } C(w_{i-n+1}^i) > 0 \\
584 \alpha(w_{i-n+1}^{i-1}) P_{\text{katz}}(w_i | w_{i-n+2}^{i-1}) & \text{otherwise}
585 \end{cases}
586 \end{equation}
587
588 \textbf{5. 插值平滑 (Interpolation)}
589
590 线性组合不同阶的模型：
591 \begin{equation}
592 P_{\text{interp}}(w_i | w_{i-2}, w_{i-1}) = \lambda_3 P(w_i | w_{i-2}, w_{i-1}) + \lambda_2
593 \quad P(w_i | w_{i-1}) + \lambda_1 P(w_i)
594 \end{equation}
595
596 其中  $\lambda_1 + \lambda_2 + \lambda_3 = 1$ 。
597
598 \subsubsection{N-gram语言模型完整实现}
599
600 \begin{lstlisting}[caption=N-gram语言模型完整实现]
601 import numpy as np
602 from collections import defaultdict, Counter
603 import re
604 import math
605 import random
606 import pickle

```

```

606
607 class NgramLanguageModel:
608     def __init__(self, n=3, smoothing='kneser_ney', alpha=0.01):
609         """
610         N-gram 语言模型
611
612         Args:
613             n: N-gram 的大小
614             smoothing: 平滑方法 ('add_one', 'add_alpha', 'good_turing', 'kneser_ney', 'interpolation')
615             alpha: Add-alpha 平滑的参数
616         """
617         self.n = n
618         self.smoothing = smoothing
619         self.alpha = alpha
620
621         # N-gram 计数
622         self.ngram_counts = defaultdict(Counter)
623         self.context_counts = defaultdict(int)
624         self.unigram_counts = Counter()
625
626         # 词汇表
627         self.vocab = set()
628         self.vocab_size = 0
629
630         # 特殊符号
631         self.start_token = '<s>'
632         self.end_token = '</s>'
633         self.unk_token = '<unk>'
634
635         # 平滑参数
636         self.good_turing_counts = Counter()
637         self.interpolation_weights = None
638
639         # 统计信息
640         self.total_ngrams = 0
641         self.total_words = 0
642
643     def preprocess_sentence(self, sentence):
644         """预处理句子，添加开始和结束标记"""
645         # 添加开始和结束标记
646         tokens = [self.start_token] * (self.n - 1) + sentence + [self.end_token]
647         return tokens
648
649     def build_vocab(self, sentences, min_count=1):
650         """构建词汇表"""
651         word_counts = Counter()
652
653         for sentence in sentences:

```

```

654         for word in sentence:
655             word_counts[word] += 1
656
657     # 过滤低频词
658     self.vocab = {word for word, count in word_counts.items() if count >=
659                     min_count}
660     self.vocab.update([self.start_token, self.end_token, self.unk_token])
661     self.vocab_size = len(self.vocab)
662
663     print(f"词汇表大小: {self.vocab_size}")
664
665     def _replace_unk(self, sentence):
666         """将未知词替换为UNK"""
667         return [word if word in self.vocab else self.unk_token for word in sentence]
668
669     def train(self, sentences):
670         """训练N-gram模型"""
671         print(f"训练{self.n}-gram语言模型...")
672
673         # 首先构建词汇表
674         if not self.vocab:
675             self.build_vocab(sentences)
676
677         # 统计N-gram
678         for sentence in sentences:
679             # 预处理句子
680             processed = self.preprocess_sentence(self._replace_unk(sentence))
681
682             # 提取N-gram
683             for i in range(len(processed) - self.n + 1):
684                 ngram = tuple(processed[i:i + self.n])
685                 context = ngram[:-1]
686                 word = ngram[-1]
687
688                 # 更新计数
689                 self.ngram_counts[context][word] += 1
690                 self.context_counts[context] += 1
691                 self.unigram_counts[word] += 1
692                 self.total_ngrams += 1
693
694             self.total_words += len(sentence)
695
696         print(f"总N-gram数量: {self.total_ngrams:,}")
697         print(f"唯一N-gram类型: {len(self.ngram_counts):,}")
698
699         # 准备平滑参数
700         if self.smoothing == 'good_turing':
701             self._compute_good_turing_counts()
702         elif self.smoothing == 'interpolation':

```

```

702         self._compute_interpolation_weights()
703
704     def _compute_good_turing_counts(self):
705         """计算Good-Turing平滑所需的频率统计"""
706         # 统计每个频率的n-gram数量
707         for context_counts in self.ngram_counts.values():
708             for count in context_counts.values():
709                 self.good_turing_counts[count] += 1
710
711     def _compute_interpolation_weights(self):
712         """计算插值权重（简化版本）"""
713         # 这里使用固定权重，实际应用中应该用EM算法优化
714         if self.n == 2:
715             self.interpolation_weights = [0.4, 0.6] # [unigram, bigram]
716         elif self.n == 3:
717             self.interpolation_weights = [0.2, 0.3, 0.5] # [unigram, bigram,
718                                                         trigram]
719         else:
720             # 对于更高阶的n-gram，使用递减权重
721             weights = [(i + 1) ** 2 for i in range(self.n)]
722             total = sum(weights)
723             self.interpolation_weights = [w / total for w in weights]
724
725     def _get_good_turing_count(self, original_count):
726         """获取Good-Turing调整后的计数"""
727         if original_count == 0:
728             return self.good_turing_counts.get(1, 1) / self.total_ngrams
729
730         next_count = original_count + 1
731         if next_count in self.good_turing_counts:
732             return (original_count + 1) * self.good_turing_counts[next_count] / self
733                 .good_turing_counts[original_count]
734         else:
735             return original_count
736
737     def get_probability(self, word, context):
738         """获取条件概率  $P(\text{word}|\text{context})$ """
739         if len(context) != self.n - 1:
740             raise ValueError(f"上下文长度应为{self.n-1}")
741
742         context = tuple(context)
743
744         # 处理未知词
745         if word not in self.vocab:
746             word = self.unk_token
747
748         if self.smoothing == 'add_one':
749             return self._add_one_probability(word, context)
750         elif self.smoothing == 'add_alpha':

```

```

749         return self._add_alpha_probability(word, context)
750     elif self.smoothing == 'good_turing':
751         return self._good_turing_probability(word, context)
752     elif self.smoothing == 'kneser_ney':
753         return self._kneser_ney_probability(word, context)
754     elif self.smoothing == 'interpolation':
755         return self._interpolation_probability(word, context)
756     else:
757         # 默认使用最大似然估计
758         return self._ml_probability(word, context)
759
760     def _ml_probability(self, word, context):
761         """最大似然估计概率"""
762         if context not in self.context_counts:
763             return 1.0 / self.vocab_size
764
765         count = self.ngram_counts[context][word]
766         total = self.context_counts[context]
767         return count / total if total > 0 else 0
768
769     def _add_one_probability(self, word, context):
770         """Add-one平滑概率"""
771         count = self.ngram_counts[context][word]
772         total = self.context_counts[context]
773         return (count + 1) / (total + self.vocab_size)
774
775     def _add_alpha_probability(self, word, context):
776         """Add-alpha平滑概率"""
777         count = self.ngram_counts[context][word]
778         total = self.context_counts[context]
779         return (count + self.alpha) / (total + self.alpha * self.vocab_size)
780
781     def _good_turing_probability(self, word, context):
782         """Good-Turing平滑概率"""
783         original_count = self.ngram_counts[context][word]
784         adjusted_count = self._get_good_turing_count(original_count)
785         total = self.context_counts[context]
786
787         if total == 0:
788             return 1.0 / self.vocab_size
789
790         return adjusted_count / total
791
792     def _kneser_ney_probability(self, word, context):
793         """Kneser-Ney平滑概率（简化版本）"""
794         D = 0.75 # 折扣参数
795
796         count = self.ngram_counts[context][word]
797         context_total = self.context_counts[context]

```

```

798
799     if context_total == 0:
800         # 回退到unigram
801         return self.unigram_counts[word] / sum(self.unigram_counts.values())
802
803     # 计算折扣概率
804     discounted_prob = max(count - D, 0) / context_total
805
806     # 计算回退权重
807     lambda_weight = (D / context_total) * len(self.ngram_counts[context])
808
809     # 回退概率 (这里简化为unigram)
810     backoff_prob = self.unigram_counts[word] / sum(self.unigram_counts.values())
811
812     return discounted_prob + lambda_weight * backoff_prob
813
814 def _interpolation_probability(self, word, context):
815     """插值平滑概率"""
816     if not self.interpolation_weights:
817         return self._ml_probability(word, context)
818
819     total_prob = 0
820
821     # 从unigram到n-gram的插值
822     for i in range(self.n):
823         if i == 0:
824             # Unigram概率
825             prob = self.unigram_counts[word] / sum(self.unigram_counts.values())
826         else:
827             # i-gram概率
828             sub_context = context[-(i):]
829             if sub_context in self.context_counts:
830                 prob = self.ngram_counts[sub_context][word] / self.
                        context_counts[sub_context]
831             else:
832                 prob = 0
833
834             total_prob += self.interpolation_weights[i] * prob
835
836     return total_prob
837
838 def get_sentence_probability(self, sentence):
839     """计算句子概率"""
840     processed = self.preprocess_sentence(self._replace_unk(sentence))
841     log_prob = 0.0
842
843     for i in range(self.n - 1, len(processed)):
844         context = processed[i - self.n + 1:i]
845         word = processed[i]

```

```

846         prob = self.get_probability(word, context)
847         if prob > 0:
848             log_prob += math.log(prob)
849         else:
850             log_prob += math.log(1e-10) # 避免log(0)
851
852     return math.exp(log_prob)
853
854
855 def get_perplexity(self, sentences):
856     """计算困惑度"""
857     total_log_prob = 0.0
858     total_words = 0
859
860     for sentence in sentences:
861         processed = self.preprocess_sentence(self._replace_unk(sentence))
862
863         for i in range(self.n - 1, len(processed)):
864             context = processed[i - self.n + 1:i]
865             word = processed[i]
866
867             prob = self.get_probability(word, context)
868             if prob > 0:
869                 total_log_prob += math.log(prob)
870             else:
871                 total_log_prob += math.log(1e-10)
872             total_words += 1
873
874     if total_words == 0:
875         return float('inf')
876
877     avg_log_prob = total_log_prob / total_words
878     perplexity = math.exp(-avg_log_prob)
879
880     return perplexity
881
882 def generate_text(self, context=None, max_length=20, temperature=1.0):
883     """生成文本"""
884     if context is None:
885         context = [self.start_token] * (self.n - 1)
886     elif len(context) < self.n - 1:
887         context = [self.start_token] * (self.n - 1 - len(context)) + context
888     elif len(context) > self.n - 1:
889         context = context[-(self.n - 1):]
890
891     generated = list(context)
892
893     for _ in range(max_length):
894         current_context = tuple(generated[-(self.n-1):])

```



```

895
896     # 获取可能的下一个词及其概率
897     if current_context in self.ngram_counts:
898         candidates = self.ngram_counts[current_context]
899
900         if not candidates:
901             break
902
903         # 计算概率分布
904         words = list(candidates.keys())
905         probabilities = []
906
907         for word in words:
908             prob = self.get_probability(word, current_context)
909             # 应用温度参数
910             probabilities.append(prob ** (1.0 / temperature))
911
912         # 归一化
913         total_prob = sum(probabilities)
914         if total_prob > 0:
915             probabilities = [p / total_prob for p in probabilities]
916
917         # 采样下一个词
918         next_word = np.random.choice(words, p=probabilities)
919
920         if next_word == self.end_token:
921             break
922
923         generated.append(next_word)
924     else:
925         break
926     else:
927         break
928
929     # 移除开始标记
930     start_index = 0
931     while start_index < len(generated) and generated[start_index] == self.
932         start_token:
933         start_index += 1
934
935     return generated[start_index:]
936
937 def evaluate_model(self, test_sentences):
938     """评估模型性能"""
939     perplexity = self.get_perplexity(test_sentences)
940
941     # 计算覆盖率
942     total_ngrams = 0
943     covered_ngrams = 0

```

```

943
944     for sentence in test_sentences:
945         processed = self.preprocess_sentence(self._replace_unk(sentence))
946
947         for i in range(len(processed) - self.n + 1):
948             ngram = tuple(processed[i:i + self.n])
949             context = ngram[:-1]
950             word = ngram[-1]
951
952             total_ngrams += 1
953             if context in self.ngram_counts and word in self.ngram_counts[
954                 context]:
955                 covered_ngrams += 1
956
957 coverage = covered_ngrams / total_ngrams if total_ngrams > 0 else 0
958
959 return {
960     'perplexity': perplexity,
961     'coverage': coverage,
962     'total_ngrams': total_ngrams,
963     'covered_ngrams': covered_ngrams
964 }
965
966 def save_model(self, filepath):
967     """保存模型"""
968     model_data = {
969         'n': self.n,
970         'smoothing': self.smoothing,
971         'alpha': self.alpha,
972         'ngram_counts': dict(self.ngram_counts),
973         'context_counts': dict(self.context_counts),
974         'unigram_counts': dict(self.unigram_counts),
975         'vocab': self.vocab,
976         'vocab_size': self.vocab_size,
977         'total_ngrams': self.total_ngrams,
978         'total_words': self.total_words,
979         'interpolation_weights': self.interpolation_weights
980     }
981
982     with open(filepath, 'wb') as f:
983         pickle.dump(model_data, f)
984
985     print(f"模型已保存到: {filepath}")
986
987 def load_model(self, filepath):
988     """加载模型"""
989     with open(filepath, 'rb') as f:
990         model_data = pickle.load(f)

```

```

991     # 恢复模型状态
992     self.n = model_data['n']
993     self.smoothing = model_data['smoothing']
994     self.alpha = model_data['alpha']
995     self.ngram_counts = defaultdict(Counter, model_data['ngram_counts'])
996     self.context_counts = defaultdict(int, model_data['context_counts'])
997     self.unigram_counts = Counter(model_data['unigram_counts'])
998     self.vocab = model_data['vocab']
999     self.vocab_size = model_data['vocab_size']
1000     self.total_ngrams = model_data['total_ngrams']
1001     self.total_words = model_data['total_words']
1002     self.interpolation_weights = model_data.get('interpolation_weights')
1003
1004     print(f"模型已从 {filepath} 加载")
1005
1006 # N-gram模型比较分析器
1007 class NgramComparator:
1008     """N-gram模型比较分析器"""
1009
1010     def __init__(self):
1011         self.models = {}
1012
1013     def add_model(self, name, model):
1014         """添加模型进行比较"""
1015         self.models[name] = model
1016
1017     def compare_perplexity(self, test_sentences):
1018         """比较不同模型的困惑度"""
1019         results = {}
1020
1021         print("困惑度比较:")
1022         print("-" * 50)
1023
1024         for name, model in self.models.items():
1025             perplexity = model.get_perplexity(test_sentences)
1026             results[name] = perplexity
1027             print(f"{name:20}: {perplexity:.2f}")
1028
1029         return results
1030
1031     def compare_generation_quality(self, contexts, num_samples=5):
1032         """比较文本生成质量"""
1033         print("\n文本生成比较:")
1034         print("-" * 50)
1035
1036         for context in contexts:
1037             print(f"\n上下文: {' '.join(context)}")
1038
1039             for name, model in self.models.items():

```

```

1040         print(f"\n{name}:")
1041
1042         for i in range(num_samples):
1043             generated = model.generate_text(context, max_length=10)
1044             generated_text = ' '.join(generated)
1045             print(f"    {i+1}. {generated_text}")
1046
1047     def analyze_coverage(self, test_sentences):
1048         """分析模型覆盖率"""
1049         print("\n覆盖率分析:")
1050         print("-" * 50)
1051
1052         for name, model in self.models.items():
1053             evaluation = model.evaluate_model(test_sentences)
1054             print(f"\n{name}:")
1055             print(f"    覆盖率: {evaluation['coverage']:.3f}")
1056             print(f"    总N-gram数: {evaluation['total_ngrams']}")
1057             print(f"    已知N-gram数: {evaluation['covered_ngrams']}")
1058
1059 # N-gram演示
1060 def demonstrate_ngram_comprehensive():
1061     """N-gram语言模型完整演示"""
1062     print("=" * 80)
1063     print("N-gram语言模型完整演示")
1064     print("=" * 80)
1065
1066 # 训练数据
1067 sentences = [
1068     ['我', '喜欢', '机器', '学习'],
1069     ['机器', '学习', '很', '有趣'],
1070     ['我', '在', '学习', '自然', '语言', '处理'],
1071     ['自然', '语言', '处理', '是', '机器', '学习', '的', '应用'],
1072     ['深度', '学习', '推动', '了', '自然', '语言', '处理', '发展'],
1073     ['我', '喜欢', '深度', '学习'],
1074     ['机器', '学习', '算法', '不断', '改进'],
1075     ['自然', '语言', '处理', '技术', '越来越', '成熟'],
1076     ['深度', '学习', '模型', '性能', '优秀'],
1077     ['我', '学习', '人工', '智能', '技术'],
1078     ['人工', '智能', '包含', '机器', '学习'],
1079     ['自然', '语言', '理解', '需要', '深度', '学习'],
1080     ['机器', '学习', '方法', '多种', '多样'],
1081     ['深度', '学习', '网络', '结构', '复杂'],
1082     ['我', '研究', '自然', '语言', '处理', '算法']
1083 ]
1084
1085 # 测试数据
1086 test_sentences = [
1087     ['我', '学习', '机器', '学习'],
1088     ['深度', '学习', '很', '有趣'],

```

```

1089     ['自然', '语言', '处理', '应用', '广泛']
1090 ]
1091
1092 print(f"训练数据: {len(sentences)} 个句子")
1093 print(f"测试数据: {len(test_sentences)} 个句子")
1094
1095 # 创建比较器
1096 comparator = NgramComparator()
1097
1098 # 训练不同的模型
1099 smoothing_methods = ['add_one', 'add_alpha', 'kneser_ney', 'interpolation']
1100
1101 print("\n1. 训练不同的模型...")
1102 for smoothing in smoothing_methods:
1103     print(f"\n训练 {smoothing} 模型...")
1104
1105     for n in [2, 3]:
1106         model_name = f"{n}-gram-{smoothing}"
1107         model = NgramLanguageModel(n=n, smoothing=smoothing, alpha=0.01)
1108         model.train(sentences)
1109         comparator.add_model(model_name, model)
1110
1111 print("\n2. 困惑度比较...")
1112 perplexity_results = comparator.compare_perplexity(test_sentences)
1113
1114 print("\n3. 生成质量比较...")
1115 test_contexts = [
1116     ['我'],
1117     ['我', '喜欢'],
1118     ['机器', '学习'],
1119     ['自然', '语言']
1120 ]
1121 comparator.compare_generation_quality(test_contexts, num_samples=3)
1122
1123 print("\n4. 覆盖率分析...")
1124 comparator.analyze_coverage(test_sentences)
1125
1126 print("\n5. 详细分析最佳模型...")
1127 # 选择困惑度最低的模型进行详细分析
1128 best_model_name = min(perplexity_results.items(), key=lambda x: x[1])[0]
1129 best_model = comparator.models[best_model_name]
1130
1131 print(f"最佳模型: {best_model_name}")
1132 print(f"最低困惑度: {perplexity_results[best_model_name]:.2f}")
1133
1134 # 分析具体句子的概率
1135 print("\n句子概率分析:")
1136 for sentence in test_sentences:
1137     prob = best_model.get_sentence_probability(sentence)

```

```

1138         print(f"'{' '.join(sentence)}': {prob:.2e}")
1139
1140     # 温度参数对生成的影响
1141     print("\n6. 温度参数对生成的影响...")
1142     context = ['我', '喜欢']
1143     temperatures = [0.5, 1.0, 1.5]
1144
1145     for temp in temperatures:
1146         print(f"\n温度 = {temp}:")
1147         for i in range(3):
1148             generated = best_model.generate_text(context, max_length=8, temperature=
                temp)
1149             print(f"    {' '.join(generated)}")
1150
1151     return comparator
1152
1153 if __name__ == "__main__":
1154     ngram_comparator = demonstrate_ngram_comprehensive()

```

Listing 3: 神经网络语言模型实现 (PyTorch)

5 总结与展望

5.1 本节课总结

本节课我们深入学习了词向量和语言模型的核心技术：

词向量技术：

- **理论基础：** 分布式假设，向量空间表示
- **经典算法：** Word2Vec (CBOW/Skip-gram), GloVe, FastText
- **评估方法：** 内在评估 (相似性, 类比), 外在评估 (下游任务)
- **应用价值：** 密集表示, 语义关系, OOV 处理

语言模型：

- **基础概念：** 序列概率建模, 困惑度评估
- **经典方法：** N-gram 统计模型, 平滑技术
- **神经方法：** 前馈神经网络, 循环神经网络
- **发展趋势：** 从统计到神经, 从固定到动态

6 课后练习

6.1 基础练习

练习 1: 词向量训练与评估

1. 使用提供的中文语料训练 Word2Vec 模型
2. 实现词汇相似性和类比任务的评估
3. 比较不同超参数设置对结果的影响
4. 可视化词向量的分布和聚类情况

练习 2: N-gram 语言模型实现

1. 实现 2-gram 和 3-gram 语言模型
2. 比较不同平滑技术的效果
3. 计算模型在测试集上的困惑度
4. 使用模型生成文本并分析质量

7 参考资料

7.1 核心论文

词向量相关:

- "Efficient Estimation of Word Representations in Vector Space" (Mikolov et al., 2013) - Word2Vec
- "GloVe: Global Vectors for Word Representation" (Pennington et al., 2014) - GloVe
- "Enriching Word Vectors with Subword Information" (Bojanowski et al., 2017) - FastText

语言模型相关:

- "A Neural Probabilistic Language Model" (Bengio et al., 2003) - 神经语言模型
- "Recurrent neural network based language model" (Mikolov et al., 2010) - RNN 语言模型
- "Attention Is All You Need" (Vaswani et al., 2017) - Transformer

课程结语

词向量和语言模型是现代 NLP 的基石技术。通过本节课的学习，我们深入理解了从传统统计方法到现代神经网络方法的发展脉络，掌握了核心算法的原理和实现。

希望同学们能够在理解基础原理的同时，关注技术发展趋势，在实践中不断深化对这些核心技术的理解和应用能力。

— 课程结束 —